Zarządzanie w systemach i sieciach komputerowych

Dr inż. Robert Wójcik

Wykład 5. Zarządzanie jakością usług w sieciach komputerowych (QoS)

5.1. Definicja jakości usług (QoS).

5.2. Podstawowe parametry QoS w sieciach komputerowych.

5.3. Metody zapewnienia Quality of Service w sieciach LAN/WAN.

5.4. Podział algorytmów kolejkowania pakietów

5.4.1. Algorytmy szeregowania bezklasowe i klasowe

5.5. Algorytmy szeregowania pakietów

5.5.1. Algorytm Priority Queuing (PQ)

5.5.2. Algorytmy sprawiedliwego kolejkowania (Fair Queuing)

5.5.3. Algorytmy ograniczania przepustowości

5.5.4. Algorytmy hierarchicznego podziału łącza

5.6. Algorytmy zapobiegania zatorom

5.7. Modele sieci komputerowych wspierających QoS

5.7.1. Cechy architektury IntServ

5.7.2. Cechy architektury DiffServ

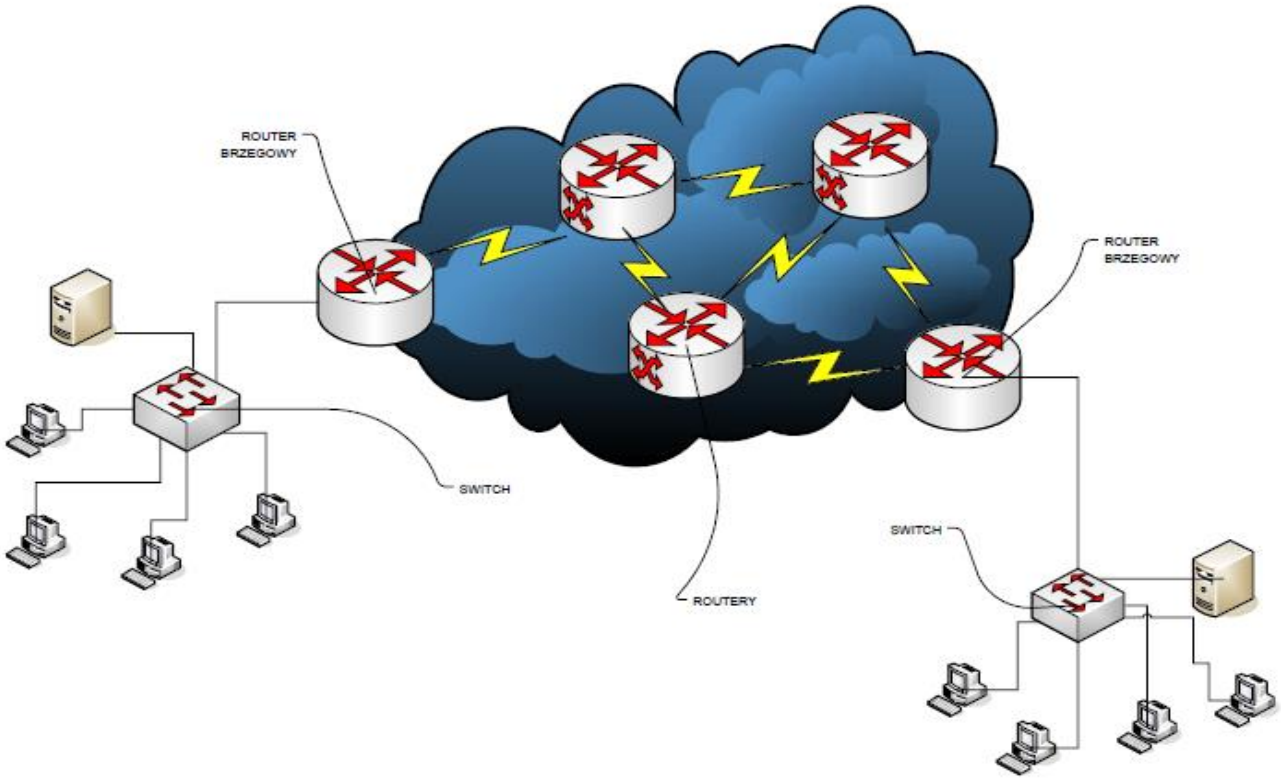
5.7.3. Współpraca modeli IntServ i DiffServ

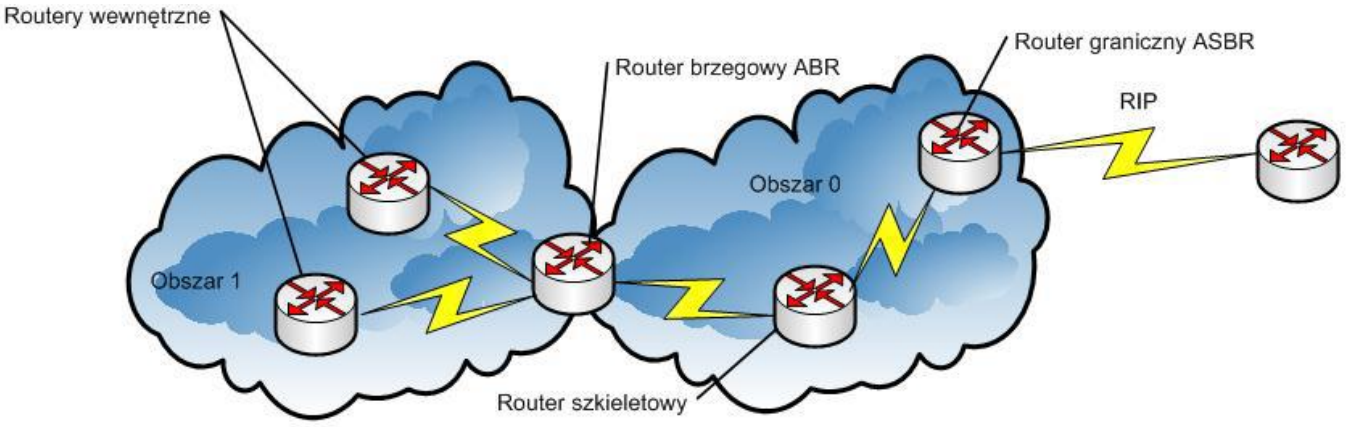
5.1. Definicja jakości usług (QoS)

Podstawowym rozwiązaniem połączenia sieci lokalnych z Internetem jest router (tzw. router brzegowy), który wykorzystuje do komunikacji sieciowej protokół IP.

Problem współdzielenia łącza transmisji danych i zasobów routerów

W sieciach lokalnych jest określona ilość użytkowników końcowych, którzy rywalizują ze sobą o dostęp do Internetu, czyli muszą współdzielić łącze internetowe.





RIP (Routing Information Protocol), ABR (Area Border Router).
ASBR (Autonomus System Boundary Router).

W wielu przypadkach przesyłane dane są znacznych rozmiarów i muszą być dostarczane w czasie rzeczywistym. Konieczność rywalizacji o dostęp do łącza transmisji danych i zasoby routerów (bufory) doprowadziła do tego, że łącze stało się tak zwanym wąskim gardłem, gdyż są znacznie większe potrzeby użytkowników współdzielących łącze niż możliwości przesłania danych.

Nowe usługi w sieciach wymagają stałego i wysokiego poziomu jakości. Dotyczy to zwłaszcza aplikacji czasu rzeczywistego, które są szczególnie wrażliwe na opóźnienia transmisji pakietów i wymagają zagwarantowanej szerokości pasma transmisyjnego. Rozwój tego typu aplikacji nakłada na sieć konieczność wprowadzania mechanizmów zapewniających transmisję danych z zachowaniem określonego poziomu pewnych parametrów w relacji od końca do końca (end-to-end), tj. z zachowaniem odpowiedniej jakości oferowanych usług - Quality of Service (QoS).

Router może przyjmować dane z Ethernetu z prędkościami rzędu megabitów na sekundę i wysyłać je łączem szeregowym o przepustowości o dwa rzędy wielkości mniejszej. W przypadku styku szybkiego łącza sieci LAN i wolnego łącza sieci WAN, siłą rzeczy dochodzi do sytuacji, gdy część pakietów nie może zostać „wepchnięta” w wąskie gardło i jest gubiona. Zastosowanie w tym miejscu sterowania przepływem danych pozwala ograniczyć straty do minimum, poprawić wykorzystanie łącza, a także, w razie potrzeby, zapewnić określonym rodzajom danych pierwszeństwo.

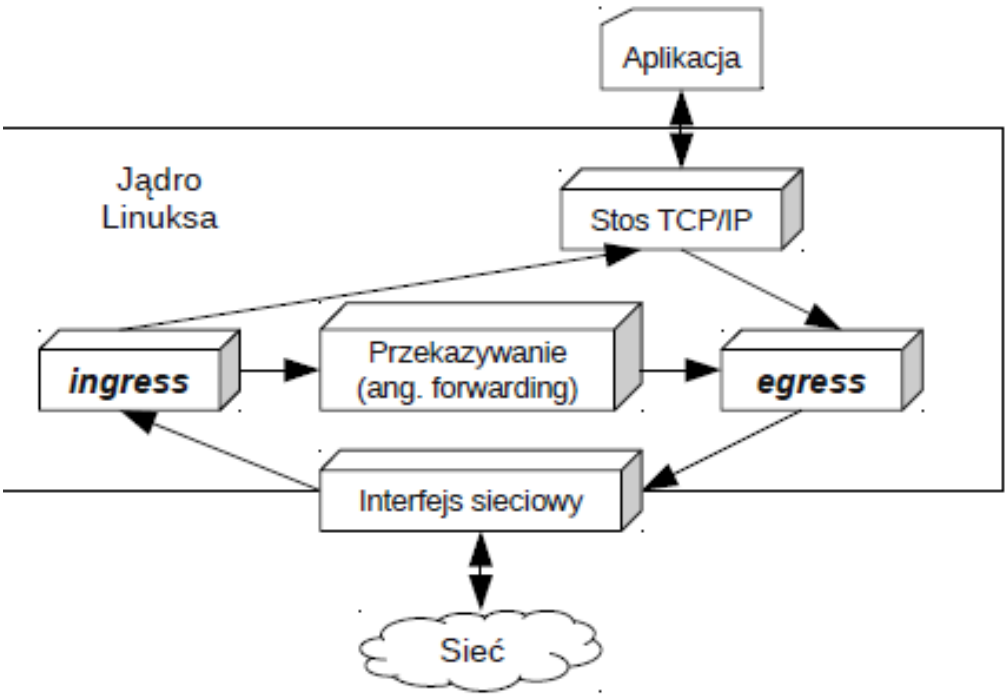
Ruch wychodzący i przychodzący

Tradycyjnie w systemie Linux istnieje fundamentalna różnica pomiędzy ruchem przychodzącym a wychodzącym. W przypadku ruchu przychodzącego, pomijając mechanizm adaptacji w protokole TCP, nie ma wbudowanych w system mechanizmów sterowania ruchem przychodzącym. Interfejs sieciowy odbiera pakiety, nie mając wpływu na ich kolejność, rozmiar ani czas dostarczenia. Ogranicza to możliwość kształtowania ruchu przychodzącego do definiowania polityki odrzucania odebranych ramek (ang. policing) oraz akceptowania (np. dostarczania do aplikacji lub rutowania pakietu dalej). W przypadku ruchu przychodzącego można implementować własne algorytmy kształtowania ruchu.

Dla ruchu wychodzącego standardowe możliwości manipulacji dostępne w środowisku Linux są znacznie większe. Możliwe jest odpowiednie kolejkowanie pakietów w celu sterowania ich przepływem w sposób gwarantujący rozładowanie zatorów i zapewnienie wymaganej jakości usług. Zmiana kolejności pakietów, często podejmowana jest przy użyciu klasyfikatorów, co pozwala m.in. nadać większy priorytet transmisjom wymagającym interaktywności, np. sesji ssh. Możliwe jest też zarządzanie przepustowością, polegające na opóźnianiu wysyłki pakietów sieciowych w celu ograniczenia wykorzystania pasma.

W Linuxsie przepływ pakietów sieciowych jest następujący (ingress – polityki zarządzania pakietami dla ruchu przychodzącego; egress – polityki kolejkowania pakietów dla ruchu wychodzącego).

students.mimuw.edu.pl/SR/prace-mgr/chimosz/gchimosz.pdf



QoS (ang. Quality of Service – jakość usługi) to, zgodnie z zaleceniem ITU-T (International Telecommunication Union) E.800, całość charakterystyk usługi telekomunikacyjnej stanowiących podstawę do wypełnienia wyrażonych i zaspokajanych potrzeb użytkownika tej usługi.

Jest wiele różnych definicji QoS.

Np. http://www.roz6.woiz.polsl.pl/~jarkarc/index.php?id=qos
http://www.roz6.woiz.polsl.pl/~jarkarc/pliki/algorytmykolejkowania.pdf

W większości przypadków można stwierdzić, że definicje te określają QoS jako zbiór mechanizmów, które mają zapewnić dostarczenie usług sieciowych o przewidywalnym poziomie jakości, poprzez zapewnienie określonych parametrów transmisji danych, w celu osiągnięcia satysfakcji użytkownika.

5.2. Podstawowe parametry QoS w sieciach komputerowych

- **Przepustowość**

Przepustowość (nazywana też pojemnością kanału) to maksymalna ilość informacji jaka może być skutecznie przesyłana przez kanał transmisyjny w danej jednostce czasu. Przepustowość mierzy się w bitach na sekundę – bits/s lub bps. Możliwe jest również wykorzystywanie jednostek pochodnych stanowiących krotność jednostki podstawowej – kbits/s, Mbits/s, Gbits/s.

Graniczna pojemność kanału wyliczona na podstawie twierdzenia Shannona:

$$C = W \cdot \log_2(1 + \frac{S}{N})$$

gdzie

W – szerokość pasma [Hz],
S/N – stosunek mocy sygnału do mocy szumu (wartości podawane w skali liniowej).

Z punktu widzenia użytkownika przepustowość jest najważniejszym wyznacznikiem sprawnej polityki QoS. W większości przypadków zwiększenie tego parametrów stanowi dostateczne rozwiązanie problemu związanego z jakością usług.

Najlepszym sposobem zarządzania przepustowością jest jej dobrze zaplanowany podział pomiędzy użytkowników lub wybrane usługi sieciowe.

- **Opóźnienie (ang. delay, latency)**

Opóźnienie to odstęp czasu pomiędzy chwilą wysłania pakietu z jednego punktu w sieci i odebrania go w innym punkcie sieci. Jest to więc czas jaki pakiet przebywa w sieci za nim dotrze do punktu docelowego.

Na sumaryczną wartość opóźnienia składają się:

1) czas propagacji w każdym z łączy należących do ścieżki przesyłania pakietu - stały w czasie parametr, który zależy od rodzaju medium transmisji danych i zastosowanej technologii,

2) czas serializacji pakietu przy opuszczaniu węzła sieci - opisany zależnością:

$$czas_serializacji = \frac{dlugosc_pakietu_w_bitach}{predkosc_interfejsu_w_b/s}$$

3) czas obróbki pakietu w węźle sieci - zależy od typu sprzętu, oprogramowania i obciążenia procesora w węźle,

4) czas opóźnienia w buforach wyjściowych w węzłach sieci - zależy od ilości napływającej informacji do węzła w danej chwili i od szybkości łącza wychodzącego.

Pierwsze dwa parametry posiadają ograniczenia fizyczne i ich poprawa nie jest możliwa poprzez stosowanie nowoczesnych technologii routingu sieciowego IP.

Trzeci parametr (czas obróbki) można ograniczyć poprzez stosowanie wydajniejszych algorytmów przetwarzania danych oraz nowoczesnych i szybkich urządzeń.

Poprawę parametru czwartego (czas opóźnienia) można uzyskać poprzez stosowanie bardziej zaawansowanych niż FIFO algorytmów kolejkowania pakietów w węzłach sieci.

Opóźnienie jest parametrem kluczowym dla aplikacji działających w czasie rzeczywistym, np. takich jak telnet, SSH, czy też w przypadku transmisji dźwięku lub sygnału wideo. Aby zmniejszyć opóźnienia w tych typach transmisji można pakiety od nich pochodzące wysyłać wcześniej niż inne, nadając im większy priorytet (kolejkowanie pakietów w buforze zgodnie z ustalonymi zasadami – priorytetami pakietów).

- **Zmienność opóźnienia (ang. jitter, delay variation)**

Zmienność opóźnienia jest to zakres, w którym zmienia się wartość opóźnienia mierzona dla pakietów należących do tego samego strumienia.

Parametr ten jest ściśle związany z zasadą pracy sieci i routingiem IP. Jak powszechnie wiadomo protokół IP jest protokołem zawodnym i nie gwarantuje tego, że pakiety dotrą do nadawcy w tej samej kolejności w jakiej zostały nadane. Dzieje się tak dlatego ponieważ przed wysłaniem dane dzielone są na pakiety, które przez sieć mogą być przesyłane różnymi trasami. W konsekwencji pakiety docierają do odbiorcy w innej kolejności niż zostały wysłane przez źródło, a więc posiadają różne opóźnienie.

Zmienność opóźnienia jest kluczowym parametrem np. w przypadku transmisji multimedialnych, w których ważna jest stała jakość transmisji z minimalnymi opóźnieniami.

Parametr ten można ograniczyć poprzez stosowanie zaawansowanych metod kolejkowania, uwzględniających zawartość pakietu, a także przez fragmentację pakietów.

Zmienność opóźnienia dość często jest również określana jako jednorodność opóźnienia.

- **Straty pakietów (ang. packet loss)**

Ze stratą pakietów mamy do czynienia wtedy kiedy jeden lub wiele pakietów nie dociera do miejsca docelowego.

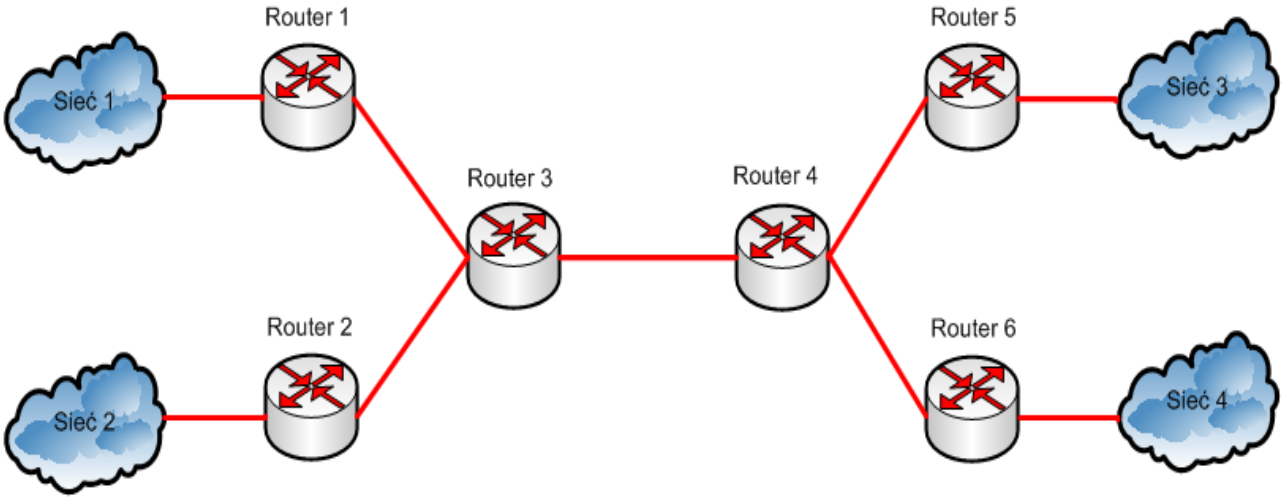
Do najważniejszych rodzajów strat pakietów należą:

- straty wynikające z błędów transmisyjnych w sieci; powstają one w sieciach o złej jakości, w momencie uszkodzenia sieci oraz w wyniku dostania się do sieci transmisyjnej niepożądanych sygnałów; ogranicza się je poprzez stosowanie łączy o wysokiej jakości;

- przepełnienie buforów wyjściowych w węzłach sieci IP; jest to zjawisko nazywane spiętrzeniem (ang. congestion); aby zminimalizować prawdopodobieństwo wystąpienia przeciążenia, należy doprowadzić do zwolnienia pracy nadajnika; niestety protokół IP nie przewiduje takich mechanizmów, możliwe jest jedynie pośrednie wpłynięcie na nadawanie danych w sesji TCP, poprzez odrzucanie pakietów aktywnej sesji.

5.3. Metody zapewnienia Quality of Service w sieciach LAN/WAN

Problem współdzielenia łącza transmisji danych można rozwiązać wprowadzając ograniczenie dostępu do sieci i korzystania z niej na zasadzie „kto pierwszy ten lepszy”, czyli obsługi metodą FIFO (First In First Out). Metoda ta powoduje, że pierwszeństwo uzyskuje ten użytkownik, który pierwszy wyśle swoje żądanie. Należy pamiętać, że łącza mają określoną przepustowość (pojemność), co determinuje całkowity czas transmisji. Prędkość ściągania plików jest taka jak przepustowość najmniejszego pośredniczącego łącza. W takiej sytuacji możliwe jest, że jeden użytkownik jest w stanie przez wiele godzin zablokować całe łącze. Podobnie może zdarzyć się podczas wysyłania plików do sieci.



Do zablokowania łącza może dojść również przy pobieraniu niewielkiej ilości danych, ale przez wielu użytkowników równocześnie. Metoda rywalizacji użytkowników musi więc skutkować negatywnymi elementami, jakimi będą straty danych i opóźnienia w ich dostarczaniu, gdyż wypełnienie możliwości przepustowych łączy powoduje dłuższą obsługę ruchu, co musi wprost prowadzić do opóźnień w przesyłaniu danych oraz przy jego całkowitym wypełnieniu do utraty przesyłanych danych.

Aby temu zapobiec, szczególnie istotne staje się zapewnienie odpowiedniego podziału przepustowości między wszystkich użytkowników i zapewnienie im tym samym odpowiedniej jakości transmisji danych.

Analizując ruch w sieciach internetowych można dojść do wniosku, że w określonej jednostce czasu przesyłane dane mają różny charakter, są różnej wielkości i mają różną intensywność, czyli transmisja danych między sieciami jest niejednorodna. W wielu przypadkach istnieje potrzeba zagwarantowania użytkownikom transmisji danych charakteryzującej się odpowiednimi parametrami jakościowymi, tj. zapewnienie wymaganej jakości usług sieciowych.

Mechanizmy QoS

- **Filtrowanie pakietów**

Najprostszy mechanizm pozwalający na sterowanie ruchem w sieciach LAN/WAN polega na filtrowaniu ruchu sieciowego. W podejściu tym następuje filtrowanie pakietów, co oznacza całkowite odrzucanie ruchu niepożądanego.

Realizowane jest to w postaci blokowania dostępu do określonych usług, co powoduje zdecydowane obniżenie ilości przetwarzanych danych, ograniczając je tylko do usług istotnych. Takie działanie, choć skuteczne, może jednak również doprowadzić do blokowania dostępu z uwagi na możliwość wystąpienia przepełnienia łącza.

Do wad filtrowania zaliczamy:

- ograniczenie dostępnych usług; filtrowanie jest mechanizmem, który pozwala poprawić jakość korzystania z jednych usług, kosztem jakości korzystania z innych; w efekcie sieć zostaje przeznaczona do z góry określonego zestawu usług, z których za jej pośrednictwem można skorzystać;

- możliwość niewykorzystywania przepustowości łącza w sytuacji kiedy ruch nieodrzucany przez filtrację nie wypełnia całej dostępnej przepustowości; w rezultacie pasmo, które może być potencjalnie dostępne pozostaje niewykorzystane;

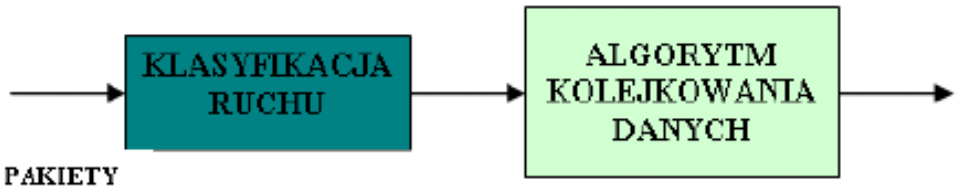
- możliwość zdominowania łącza jednego rodzaju ruchem, który jest akceptowany.

Filtrowanie powinno być stosowane tylko jako rozwiązanie tymczasowe, gdyż jest to rozwiązanie, które nie nadaje się zbyt dobrze do zapewnienia odpowiedniej jakości usług w sieciach LAN/WAN.

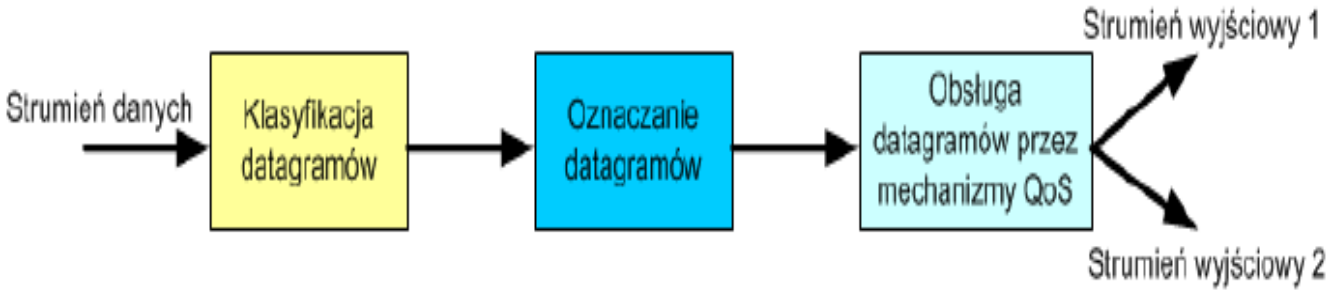
- **Klasyfikacja i kolejkowanie (szeregowanie) pakietów**

Inne podejście opiera się na wykorzystaniu mechanizmu kolejkowania danych, napływających do bufora routera, z możliwością wstępnej klasyfikacji pakietów i ich podziału na kilka kolejek, które reprezentują odpowiednie klasy ruchu.

Model **kształtowania ruchu** w routerach.

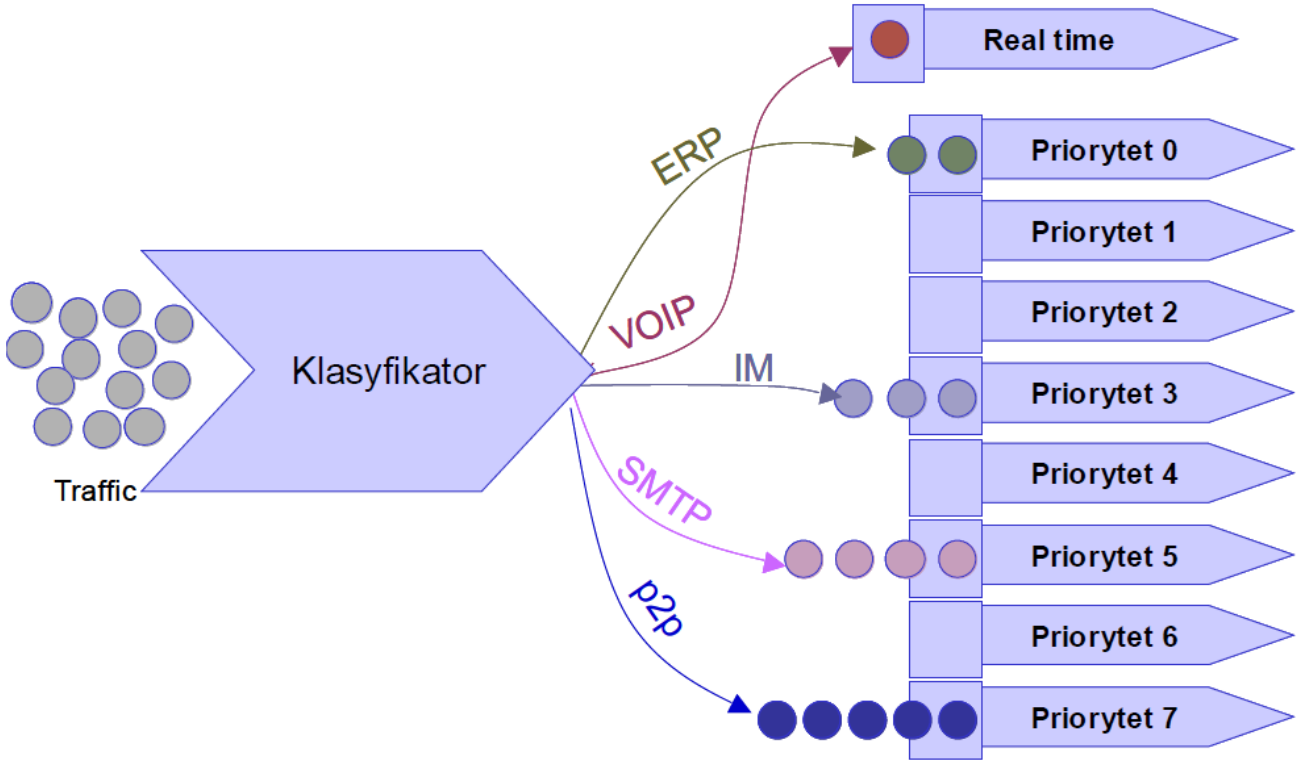


Klasyfikacja i oznaczanie pakietów



Np. http://www.roz6.woiz.polsl.pl/~jarkarc/index.php?id=qos
http://www.roz6.woiz.polsl.pl/~jarkarc/pliki/algorytmykolejkowania.pdf

Podział na przykładowe klasy ruchu: www.clico.pl/b/t/no_12/wsd_tekst.pdf



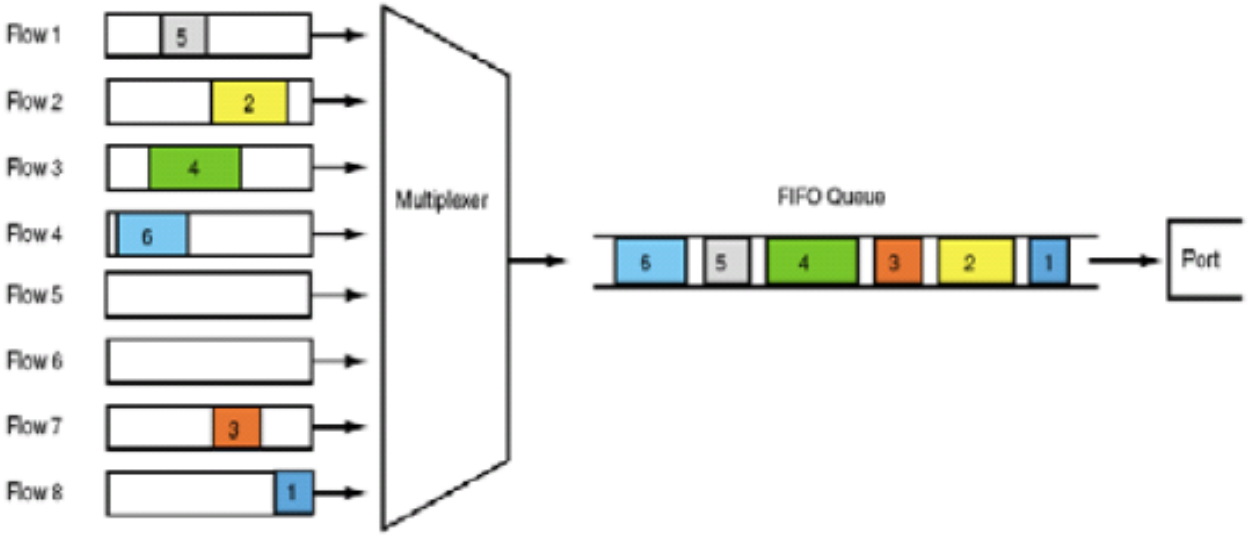
5.4. Podział algorytmów kolejkowania pakietów

Dane wpływające do routera sieciowego umieszczane są w znajdującym się w nim buforze, zanim zostaną przesłane dalej lub przetworzone. W przypadku węzłów sieciowych ich zadaniem jest *wygładzanie prędkości napływających danych, w warunkach różnych przepustowości łącz sieciowych* w danej sieci, między którymi ten węzeł pośredniczy. Algorytmy kolejkowania danych w tego typu urządzeniach sieciowych zajmują się sposobem obsługi danych przechodzących przez bufor. Dzielą one bufor na pewną ilość kolejek, które są następnie obsługiwane według określonego algorytmu postępowania (**algorytm FIFO**, **algorytmy szeregowania pakietów**, **algorytmy zapobiegania zatorom**).

Klasyczny algorytm FIFO

W tym przypadku pakiety należące do odpowiednich strumieni danych są ustawiane w buforze w ***jednej kolejce*** zgodnie z regułą FIFO.

Pakiety, które pierwsze wchodzą do bufora, pierwsze go opuszczają, w kolejności takiej, w jakiej do niego wpłynęły.



Efekt działania tego algorytmu na przepływ danych w sieciach LAN/WAN jest związany z ograniczoną objętością bufora, który w momencie zapełnienia odrzuca przychodzące pakiety, do momentu, aż nie zostanie w nim zwolnione jakiekolwiek miejsce. Mechanizm FIFO może powodować powstawanie dużych opóźnień, co ma bardzo niekorzystny wpływ na ruch aplikacji czasu rzeczywistego, które rośnie wraz ze stopniem zapełnienia bufora przez napływające pakiety. Algorytm ten traktuje wszystkie pakiety równorzędnie i nie daje możliwości ustalenia żadnych specjalnych warunków (priorytetów) dla danego wyodrębnionego ruchu.

Są jednak pewne zalety szeregowania typu FIFO:

- mechanizm ten jest bardzo prostą techniką, nie wymagającą specjalistycznej wiedzy i zużywającą najmniejsze zasoby sprzętowe;

- opóźnienie powstające w kolejce jest tu także bardzo przewidywalne, gdyż wynika ono ze stopnia jej zapełnienia;

Podsumowując, algorytm ten w wyraźny sposób nie dostarcza żadnych mechanizmów kształtowania ruchu, przychodzące pakiety przekazuje dalej w sposób taki, w jaki do niego dotarły. Z powyższego faktu wynikła konieczność opracowania innych algorytmów, umożliwiających sterowanie przepływem danych z uwzględnieniem klas ruchu i priorytetów.

Wśród algorytmów kolejkowania ruchu sieciowego, które zostały opracowane jako rozwinięcie algorytmu „First In First Out", można wyróżnić różne algorytmy szeregowania pakietów.

Algorytmy szeregowania pakietów

Algorytmy szeregowania pakietów są rozpatrywane z punktu widzenia podziału zasobów w sytuacjach, gdy dochodzi do przeciążenia sieci. Nadchodzące pakiety są w tych mechanizmach buforowane w odpowiednio zdefiniowanych kolejkach. W przypadku jednak, gdy nadmiar napływających danych jest długotrwały, pojemność kolejki jest niewystarczająca do zachowania wszystkich przychodzących pakietów, zatem nadmiarowa część musi zostać odrzucona. W przypadku sesji TCP spowoduje to ponowną retransmisję tychże pakietów, co może jeszcze wzmocnić zjawisko przeciążenia. Zadaniem algorytmów szeregowania zadań jest zmiana kolejności pakietów w buforze w sytuacjach przeciążenia sieci, tak aby osiągnięta została określona jakość obsługi.

Algorytmy zapobiegania przeciążeniom

Algorytmy te przeciwdziałają zapełnianiu się bufora urządzenia sieciowego, co może prowadzić do zatoru łącza i konieczności odrzucania pakietów.

5.4.1. Algorytmy szeregowania bezklasowe i klasowe

Wymienione rodzaje algorytmów kolejkowania (szeregowania) można niezależnie podzielić na:

- algorytmy bezklasowe,
- algorytmy oparte o klasy ruchu.

Z punktu widzenia działania tych algorytmów istotne są takie pojęcia jak:

- dyscypliny kolejkowania (główne algorytmy kolejkowania),

- filtry (klasyfikatory),

- klasy.

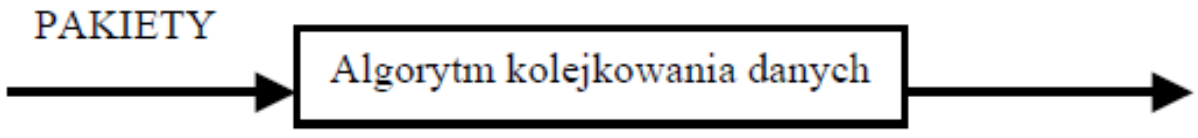
Dyscypliny kolejkowania (ang. queueing disciplines) są elementem wykonawczym kontroli pasma w routerze (np. Linuxie). Znajdują się one w pierwszej warstwie modelu TCP/IP, jednak mając dostęp do całego pakietu mogą sięgać do nagłówków protokołów wyższych poziomów. Ukrywają one w sobie algorytmy planowania i kształtowania ruchu dla ruchu wychodzącego lub określania polityki (odrzucania lub akceptowania pakietów) dla ruchu przychodzącego.

Algorytmy bezklasowe

Algorytmy bezklasowe charakteryzują się tym, że działają samodzielnie. Modelowo w ramach obsługi w obrębie jednego interfejsu sieciowego nie można połączyć ich z innymi algorytmami kolejkowania. Pakiet po obsłużeniu przez dany algorytm w buforze, opuszcza interfejs sieciowy i zostaje przekazany dalej.

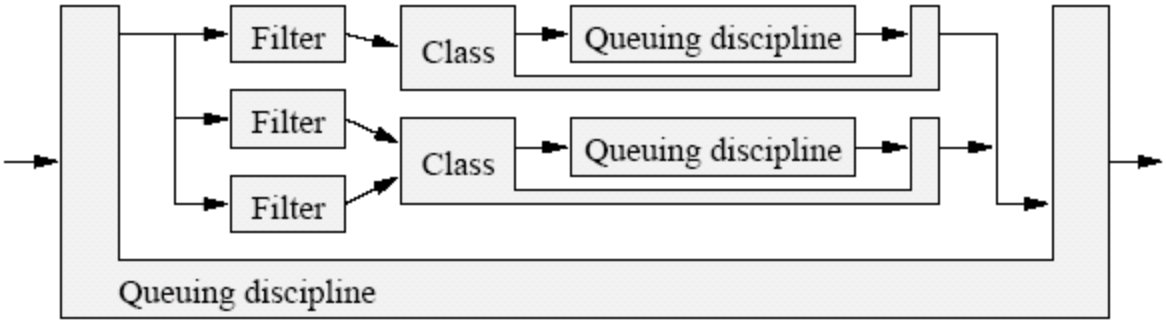
W tym przypadku kryterium bezklasowości sprowadza się do nieudostępniania mechanizmu podłączania kolejnych algorytmów kolejkowania (dyscyplin kolejkowania) do wyjścia danej kolejki bezklasowej.

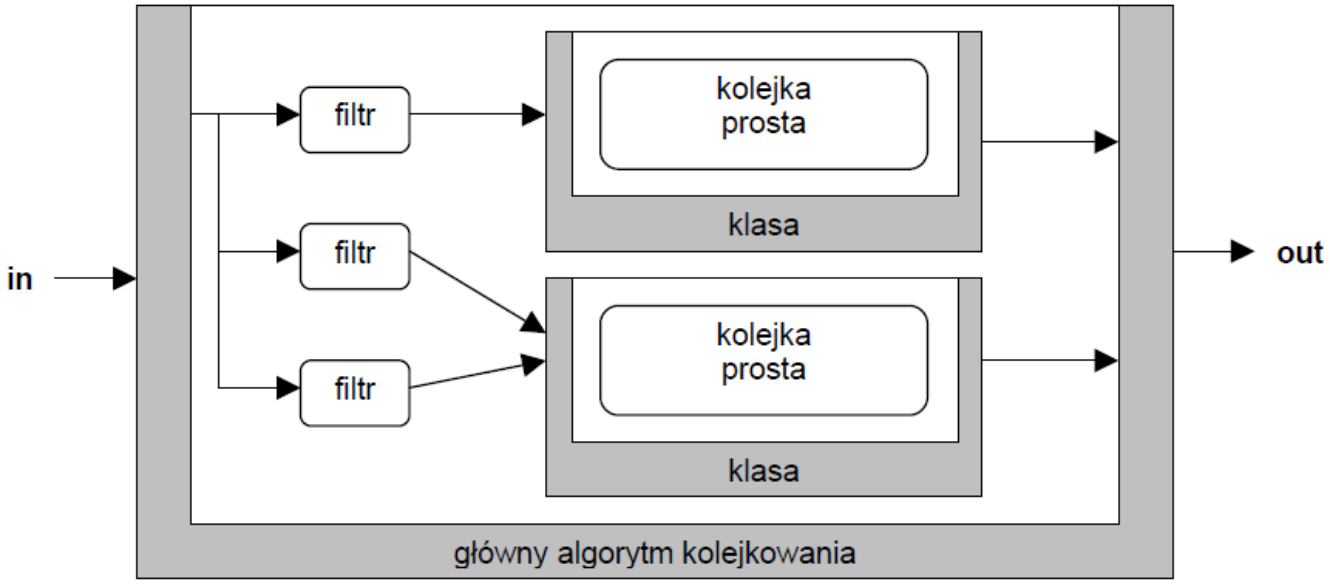
Bezklasowe algorytmy kolejkowania zbudowane są w oparciu o proste kolejki, które jedynie przyjmują i wydają pakiety, ewentualnie odraczając zwrócenie w czasie. Najlepszym przykładem jest najczęściej stosowna kolejka prosta FIFO.



Algorytmy klasowe

Algorytmy klasowe posiadają możliwość połączenia ich z innymi algorytmami w obrębie jednego interfejsu sieciowego, także opartymi o klasy, bądź bezklasowymi.



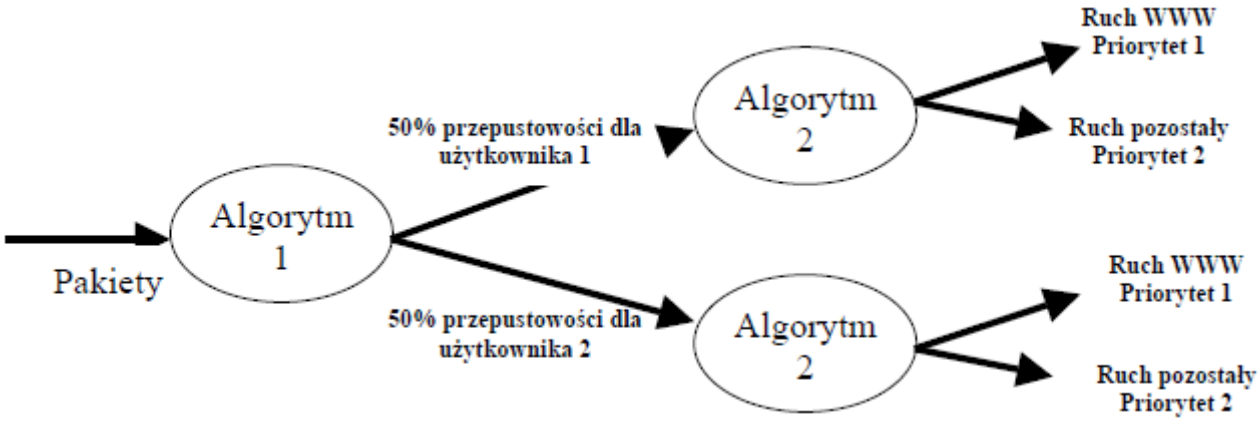


Charakterystyczną cechą konfiguracji algorytmów (dyscyplin) z klasami jest budowanie drzewa dyscyplin. Pakiety pochodzące od aplikacji są kierowane w routerze (serwer Linuxa) do głównego algorytmu kolejkowania, znajdującego się w korzeniu drzewa. Algorytm w korzeniu drzewa decyduje o dodaniu pakietu do swojej wewnętrznej kolejki lub o przekazaniu go do algorytmu jednego z dzieci w drzewie dyscyplin. Proces ten powtarza sie aż do dodania pakietu do kolejki wewnętrznej lub dotarcia do liścia drzewa. W ten sposób powstaje logiczna struktura, która wewnętrznie przypisze pakiet do określonej kolejki prostej, co definiuje kolejność pobrania pakietu.

Poszczególne klasy grupują pakiety w oparciu o różnego typu kryteria, którymi mogą być adresy warstwy sieciowej, numery portów warstwy transportowej, typy aplikacji przez jakie zostały wygenerowane oraz inne. Każdej klasie przypisywana jest też przeznaczona dla niej przepustowość. Klasy potomne mogą dziedziczyć przepustowość od swoich przodków, a także pożyczać ją pomiędzy sobą. Dodatkowo każda klasa ruchu może posługiwać się własnym algorytmem kolejkowania bezklasowego, przykładowo którymś z wymienionych powyżej.

Korzeń jest jedynym elementem drzewa, do którego odwołuje się podsystem sieciowy wysyłający pakiety pochodzące z aplikacji do sieci. Przekazywanie pakietu pomiędzy węzłami drzewa jest dzięki temu ukryte i dyscyplina może być obsługiwana dokładnie tak samo jak bezklasowa.

Ruch wchodzący do algorytmu znajdującego się w korzeniu drzewa zostaje skierowany do odpowiedniej kolejki, po obsłudze zgodnie z jego założeniem zostaje następnie skierowany do kolejnego algorytmu, definiującego kolejną klasę, gdzie także zostają utworzone kolejki według założenia tego następnego algorytmu i zgodnie z jego mechanizmami obsłużony. W efekcie, jeśli system zarządzania pakietami pobiera pakiet z określonej kolejki, to pakiet ten będzie pochodził z jednej z dyscyplin kolejkowania utworzonych w wyniku złożenia kilku algorytmów priorytetowania klasowego.



Dzięki takiemu mechanizmowi można zbudować zintegrowany algorytm, który jest w stanie zapewnić pożądaną obsługę ruchu sieciowego na kilka różnych sposobów.

Przykładowo: potrzebą użycia takich algorytmów może być sytuacja, kiedy należy najpierw przepustowość podzielić według danych założeń między użytkowników (np. każdemu dokładnie po równo), czym zajmuje się jeden algorytm, następnie w ramach każdego użytkownika kolejny algorytm zajmuje się podziałem jego ruchu nadając pierwszeństwo dostarczenia danych według pewnych ustalonych priorytetów.

Najczęściej używanymi dyscyplinami kolejkowania opartymi o klasy są: metoda CBQ (ang. Class Based Queueing) oraz metoda HTB, będąca jej szczególnym przypadkiem (nieco prostsza w działaniu), oparta na hierarchicznych kubełkach żetonów (ang. Hierarchichal Token Bucket). Pierwsza z nich pozwala zbudować drzewo dowolnych dyscyplin, druga z kolei wykorzystuje koncepcję filtra opartego o kubełek żetonów, tworząc hierarchię kubełków z możliwością przekazywania niewykorzystanych żetonów w dół drzewa.

5.5. Algorytmy szeregowania pakietów

Algorytmy kolejkowania danych w sieciach LAN/WAN zajmują się odpowiednią obsługą użytkowników, rozumianą jako podział zasobów łącza z siecią rozległą według ustalonych zasad między tych użytkowników. Algorytmy kolejkowania zajmujące się takim podziałem, nazywa się algorytmami szeregowania pakietów.

Głównym ich zadaniem jest w praktyce podział parametru, jakim jest przepustowość w sytuacjach zatorów sieci, a więc działają one w momencie gdy do przeciążeń już dochodzi. Jeśli przeciążenie sieci nie występuje, podział przepustowości nie jest tak istotny, gdyż pozostaje wolna część łącza i nie istnieje wtedy konieczność jego podziału. Konieczność ta powstaje w momencie, kiedy wykorzystywana jest całkowita przepustowość łącza sieciowego, przez co nie mogą zostać obsłużone kolejne żądania.

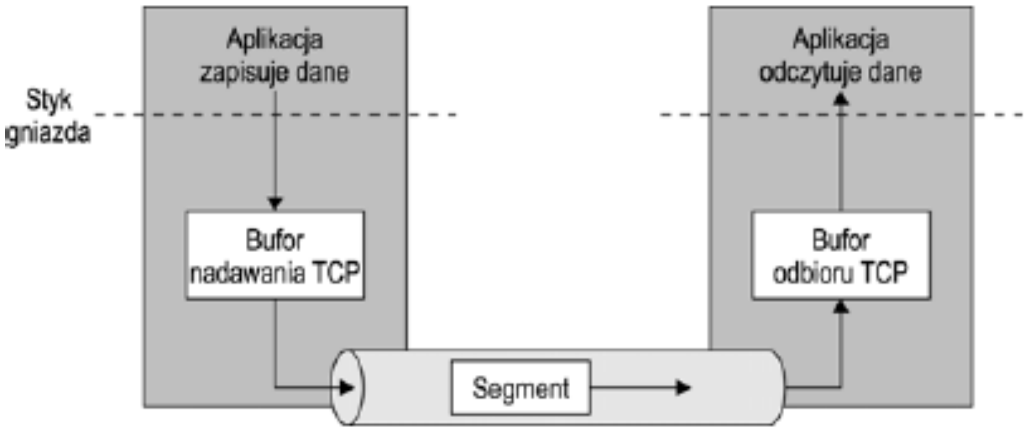
Algorytmy te wykorzystują jedną z właściwości protokołu TCP, który pozwala na dostosowanie prędkości przesyłania danych od nadawcy do odbiorcy, do warunków panujących w sieci. Zadaniem tego mechanizmu jest wysyłanie pakietów w zależności od rozmiaru bufora u odbiorcy, co ma zagwarantować, iż bufor po stronie odbiorcy nie zostanie przepełniony i pakiety nie będą tracone.

Okno nadawania i odbioru

Aby dane były dostarczane w sposób wiarygodny, klient musi otrzymać od odbiorcy potwierdzenie ACK każdego wysłanego przez siebie segmentu danych. Ponieważ klient musi czekać na potwierdzenie od odbiorcy przed nadaniem kolejnego segmentu, proces ten może prowadzić do wolnego przesyłania danych oraz niepełnego wykorzystania zasobów sieciowych. Aby zminimalizować czas jałowy sieci i zapewnić wydajne i wiarygodne przesyłanie danych, TCP wykorzystuje ideę *okien przesuwnych* (sliding window). W oknie przesuwnym przed oczekiwaniem na potwierdzenie ACK nadawanych jest kilka segmentów danych. Liczba segmentów, jaką nadawca może wysłać w określonym połączeniu, zanim otrzyma potwierdzenie ACK od odbiorcy, wskazujące iż ten otrzymał przynajmniej jeden segment danych, nosi nazwę ***okna nadawania*** (send window). Okno to ma stały rozmiar, a wszystkie segmenty mieszczące się wewnątrz okna można nadać nie czekając na potwierdzenie.

Gdy nadawca otrzyma potwierdzenie odbioru pierwszego segmentu okno nadawania przesuwa się i kolejny segment zostaje wysłany. Jeśli nadawca otrzyma potwierdzenie odbioru kilku segmentów, na przykład trzech, okno nadawania rozszerza się i przesuwa tak, że zostają wysłane kolejne trzy segmenty.

Bufory nadawania i odbioru



Jednakże liczba segmentów, które można wysłać, jest zależna od ***okna odbioru***. Po stronie odbiorczej proces aplikacji odczytuje dane z bufora z określoną prędkością, wobec czego rozmiar okna odbioru jest zależny od szybkości, z jaką dane są odczytywane. Gdy odbiorca wysyła potwierdzenie segmentów danych do klienta, razem z potwierdzeniem ACK ogłaszany jest rozmiar okna. Klient wysyła następnie segmenty z okna nadawania tak, by nie przepełnić okna odbioru po stronie odbiorcy.

Aby uniknąć przeciążenia, klient musi „pamiętać” rozmiar okna odbioru. Ponadto rozmiar okna nadawania jest zmniejszany w zależności od poziomu przeciążenia. Takie zmniejszone okno nadawania nosi nazwę **limitu okna podczas przeciążenia** lub **okna przeciążenia**.

Okno przeciążenia (nadawania) jest mechanizmem kontroli przeciążeń, wymuszanym przez nadawcę i opartym na szacunku przeciążenia sieci według nadawcy. Z drugiej strony okno odbioru jest mechanizmem kontrolnym stosowanym przez odbiorcę i opartym na ocenie dostępnej objętości wolnego miejsca w buforze. Dopuszczalny **rozmiar okna transmisji danych** jest zawsze mniejszą z dwóch wartości: okna odbioru ogłoszonego przez odbiorcę i okna przeciążenia (nadawania) przyjętego przez nadawcę.

W stanie, gdy nie występują przeciążenia, rozmiar okna przeciążenia (nadawcy) jest równy rozmiarowi okna odbiorcy. Jednakże w razie zatorów rozmiar okna nadawcy jest zmniejszany. Do oszacowania rozmiaru okna przeciążenia protokół TCP stosuje następującą strategię:

1. Redukcja okna przeciążenia (nadawania) o połowę po każdej utracie segmentu.
2. Jeśli straty dalej występują, rozmiar okna zmniejszany jest wykładniczo.
3. W ostateczności transmisja zostaje ograniczona do pojedynczych segmentów, a dopuszczalne czasy oczekiwania przed retransmisją są nadal podwajane.

Dzięki opisanemu mechanizmowi możliwe jest sterowanie prędkością wysyłanych danych od nadawcy, i tą właściwość wykorzystują algorytmy szeregowania pakietów.

Po utworzeniu odrębnej kolejki o określonej objętości dla danego rodzaju ruchu sieciowego, jeśli ta kolejka zostanie zapełniona, następuje odrzucanie kolejnych napływających do niej pakietów, a przez to zmniejszenie prędkości ich wysyłania przez nadawcę.

Wśród algorytmów szeregowania pakietów (zarządzania dostępem do łącza w warunkach występowania przeciążeń), które umożliwiają rozwiązanie problemów związanych z algorytmem kolejkowania FIFO, można wyróżnić:

- **algorytm kolejkowania priorytetowego**,

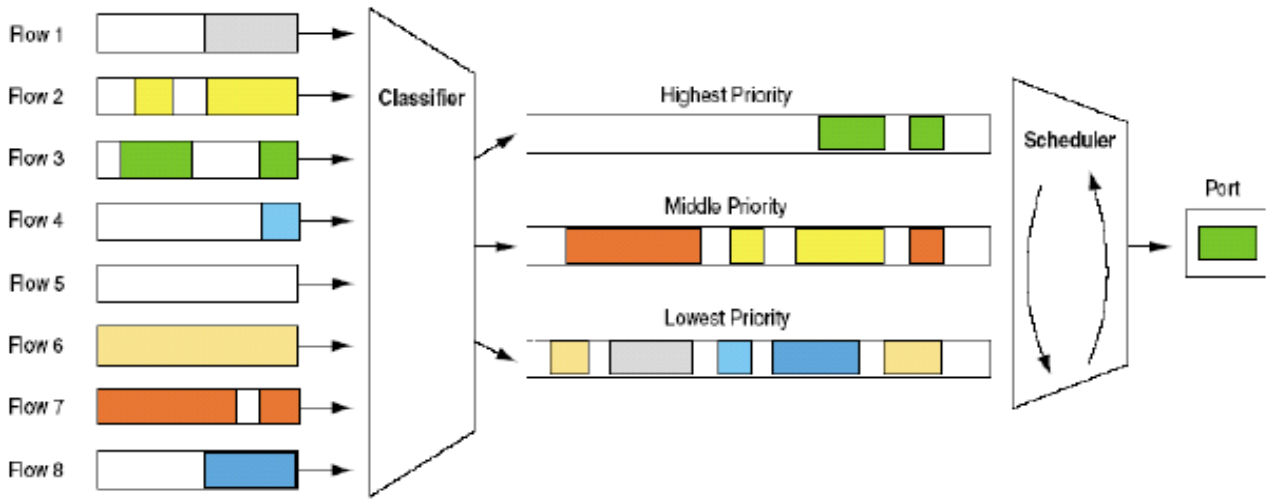
- **algorytmy sprawiedliwego kolejkowania,**

- **algorytmy ograniczania przepustowości,**

- **algorytmy hierarchicznego podziału łącza**.

5.5.1. Algorytm Priority Queuing (PQ)

Najprostszym algorytmem szeregowania pakietów, będącym niewielkim rozwinięciem algorytmu „First In First Out” i pozwalającym na bardzo ograniczony podział łącza, jest algorytm Priority Queuing (PQ). Umożliwia on podział ruchu na trzy lub cztery grupy kierowane do trzech lub czterech różnych kolejek ustawionych hierarchicznie, priorytetyzując w ten sposób jeden ruch względem innego.

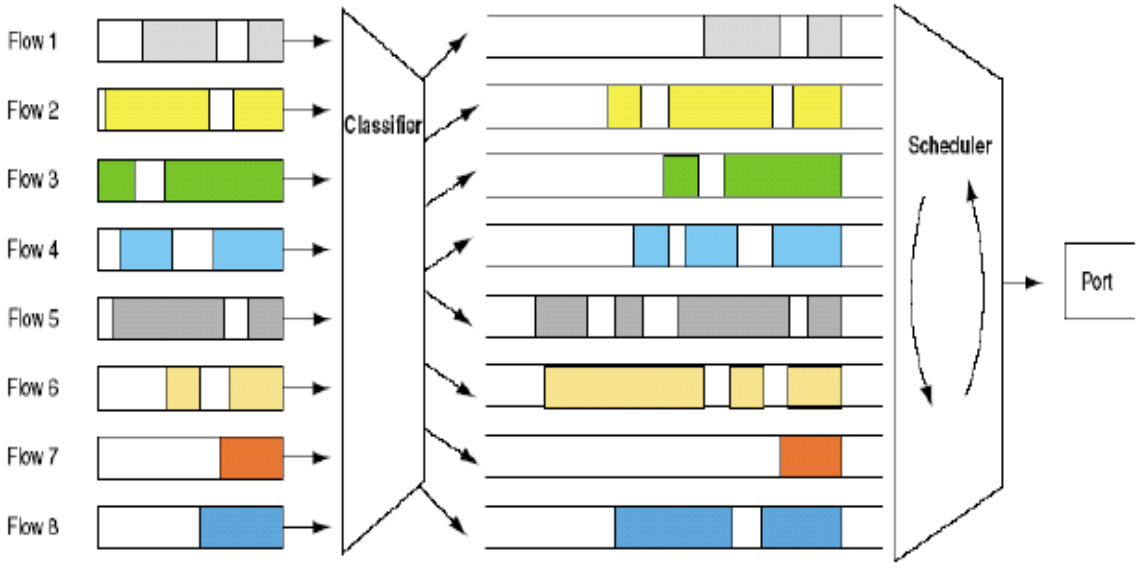


Każda kolejka priorytetowa jest z kolei obsługiwana zgodnie z metodą „First In First Out”, jakkolwiek pakiet nie może opuścić danej kolejki dopóki kolejka hierarchicznie wyższa nie zostanie opróżniona. Poza możliwością dość ograniczonej priorytetyzacji algorytm ten posiada wszystkie wady i zalety algorytmu FIFO.

W praktyce algorytmy kolejkowania priorytetowego tworzą wiele kolejek dla każdego interfejsu sieciowego i każdej z takich kolejek przypisują odpowiedni priorytet. Urządzenia korzystające z kolejek priorytetowych najpierw decydują, do których kolejek przypisać jaki rodzaj ruchu poprzez sprawdzanie pewnych reguł skonfigurowanych wcześniej. Następnie algorytm przepuszcza jako pierwsze pakiety pochodzące z kolejek o wysokim priorytecie, a dopiero kiedy kolejki te zostaną opróżnione, obsługuje kolejno bufory o niższych priorytetach. Oznacza to, że pewne przepływy będą lepiej traktowane od innych.

5.5.2. Algorytmy sprawiedliwego kolejkowania (Fair Queuing)

W celu wyeliminowania niedoskonałości algorytmu FIFO zaproponowano inne rozwiązania oparte o algorytmy „sprawiedliwego kolejkowania” (ang. Fair Queuing), w których utrzymywane są oddzielne kolejki dla pakietów z określonych źródeł. Kolejki te zostają następnie obsługiwane metodą „Round-Robin”, która zakłada cykliczne opróżnianie kolejek po jednym pakiecie (co pokazano na kolejnym rysunku).



Powstało bardzo wiele algorytmów sprawiedliwego kolejkowania, których główną cechą jest „sprawiedliwy” podział łącza transmisji danych. Przy czym ta sprawiedliwość w różnych algorytmach może być różnie rozumiana, może to być:

- równy podział przepustowości między nadawców (klasyfikowanie do kolejki pakietów z tym samym adresem źródłowym),

- równy podział przepustowości między odbiorców (klasyfikowanie do kolejki pakietów z tym samym adresem docelowym),

- równy podział przepustowości między strumienie danych (klasyfikowanie do kolejki pakietów z tym samym adresem źródłowym i docelowym, protokołem i numerem portu źródłowego i docelowego).

Najpopularniejsze algorytmy kolejkowania danych z rodziny „sprawiedliwych”, to poza samym Fair Queuing (FQ):

- SFQ (ang. Stochastic Fairness Queuing),
- WFQ (ang. Weighted Fair Queuing),
- WRR (ang. Weighted Round Robin),
- DRR (ang. Deficit Round Robin).

Powstałe w tej klasyfikacji kolejki są następnie obsługiwane w sposób cykliczny za pomocą mechanizmu „Round-Robin”. Przy czym każda kolejka ma swoją określoną pojemność, po której przekroczeniu, kolejne nadchodzące pakiety zostaną odrzucone, do momentu zwolnienia w niej miejsca.

Niektóre algorytmy kolejkowania pozwalają ponadto na przypisywanie określonym kolejkom wag liczbowych, co pozwala na to, że mimo iż wszystkie kolejki są obsługiwane cyklicznie, niektóre z nich, te o wyższej wadze, mogą być obsługiwane częściej. Przykładem takich algorytmów jest algorytm WFQ (ang. Weighted Fair Queuing).

SFQ (Stochastic Fairness Queueing)

Algorytm SFQ to prosta i posiadająca niewielkie wymagania obliczeniowe odmiana algorytmu „sprawiedliwego kolejkowania” (fair queueing). Kolejka jest w tym algorytmie rozpatrywana jako składająca się z ciągów pakietów zwanych konwersacjami (conversations) lub strumieniami (flows). Do jednej konwersacji należą pakiety posiadające takie same parametry: adresy źródłowe i docelowe oraz protokół w nagłówku IP. Przypisanie do odpowiedniej kolejki odbywa się w oparciu o wartość funkcji haszującej z tych parametrów. Co pewien czas funkcja haszu jest zmieniania, aby zmniejszyć wpływ konfliktów funkcji haszu na przydział strumieni pakietów o różnych parametrach do tych samych kolejek, tj. nierównomierność transmisji (stąd stochastic). Każda z konwersacji (kolejek) jest obsługiwana sprawiedliwie i cyklicznie (round-robin), to znaczy w każdym przebiegu wysyłany jest jeden pakiet z każdego ciągu. Konsekwencją takiego podejścia jest to, że najszybciej obsługiwane są konwersacje „krótkie”, czyli stanowiące małe obciążenie dla sieci. W przypadku przepełniania kolejki gubione są pakiety z jej końca (tail-drop). SFQ nie potrafi jednak odróżnić danych interaktywnych od masowych. Jeśli zachodzi taka potrzeba, selekcję należy przeprowadzić wcześniej, używając SFQ jako algorytmu kolejkowania dla podklas HTB lub CBQ przeznaczonych dla ruchu masowego. SFQ zapewnia tylko sprawiedliwe współdzielenie łącza przez kilka aplikacji, zwiększa przewidywalność czasów wędrówki pakietów konwersacji o dużym natężeniu i zapobiega przejęciu całego pasma przez jedną aplikację generującą duży ruch (link takeover).

Zaletą algorytmów sprawiedliwego kolejkowania w stosunku do algorytmu „First In First Out”, jest możliwość równego podziału łącza, bądź między nadawców, bądź odbiorców, bądź sesje danych. Taki podział zapobiega możliwościom powstania zatorów łącza.

Wadą tych rozwiązań jest natomiast brak możliwości ustalania określonej przepustowości dla danego ruchu. Z wyjątkiem algorytmów ważonych, nie ma możliwości priorytetyzacji danego strumienia danych. Również, jeśli któraś z powstałych kolejek będzie posiadała większy stopień zapełnienia, czas ich obsługi zwiększy się powodując wzrost opóźnienia.

5.5.3. Algorytmy ograniczania przepustowości

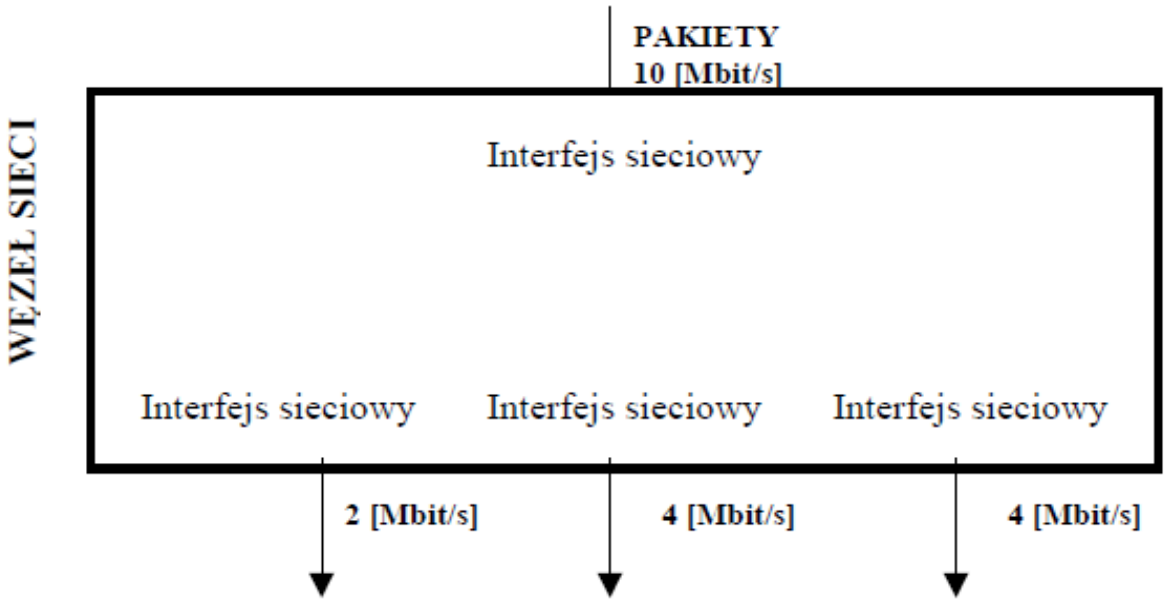
Algorytmy sprawiedliwego kolejkowania pozwalają na równy podział przepustowości, przy czym może on być pomiędzy nadawcami, odbiorcami bądź sesjami danych.

Istnieje jednak czasem potrzeba rozdziału przepustowości pomiędzy podsieci na różne, a zarazem bardzo konkretne wartości, bądź też spowolnienia transmisji danych wychodzących z węzła w innych celach. Takim podziałem zajmują się *algorytmy ograniczania przepustowości*, w tym:

- algorytm oparty o model „cieknącego wiadra” (Leaky Bucket),
- algorytm oparty o model „wiadra z żetonami” (Token Bucket), wykorzystujący do kształtowania ruchu koncepcję żetonów.

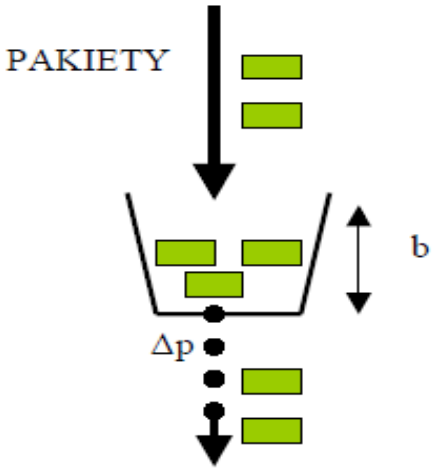
Istotą działania tych algorytmów jest ograniczenie przepływu pakietów tak, aby opuszczały one bufor interfejsu sieciowego nie szybciej niż ustalona wartość, lub zamiana nieuregulowanego napływu pakietów z aplikacji na ruch uregulowany do łącza sieciowego.

Przykładowo: potrzebą użycia tego typu algorytmów, może być sytuacja, gdy istnieje konieczność rozdzielenia przepustowości łącza o wielkości 10 [Mbit/s] pomiędzy trzy podsieci, gdzie dwie podsieci mogą transmitować dane nie szybciej niż 4 [Mbit/s], a jedna podsieć nie szybciej niż 2 [Mbit/s].

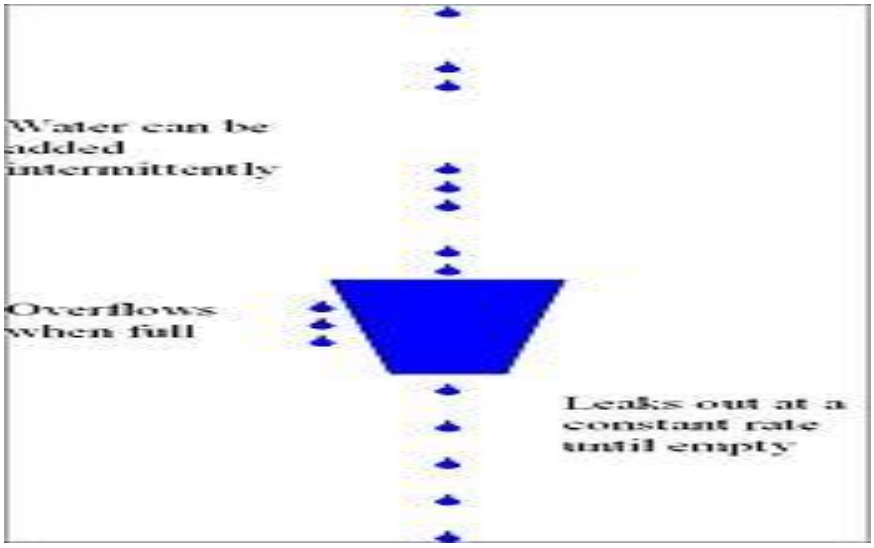


Model „cieknącego wiadra” (Leaky Bucket)

W modelu tym, nadchodzące z aplikacji pakiety są buforowane w „wiadrze” o określonej głębokości b. Jeśli wypełnią one dopuszczalny rozmiar wiadra b, kolejne nadchodzące pakiety są odrzucane, do momentu zwolnienia miejsca w wiadrze.



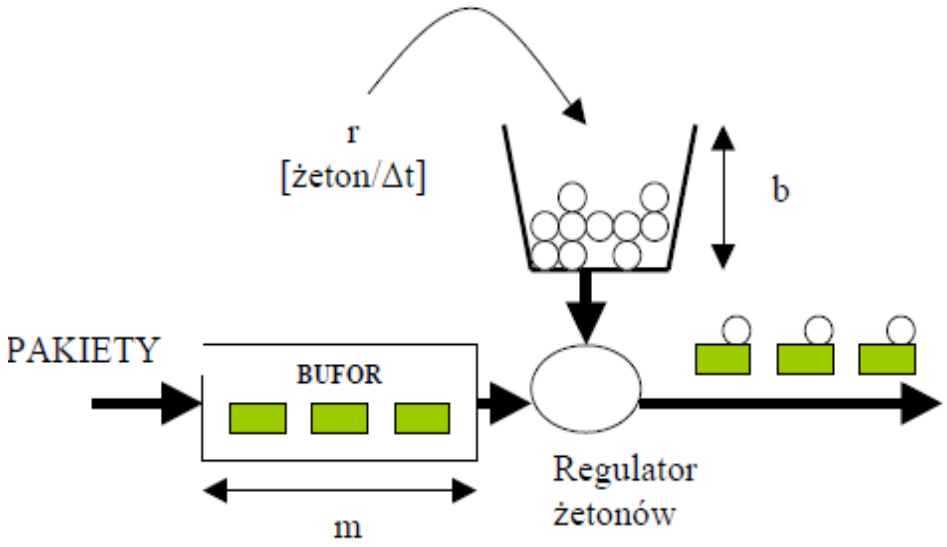
Pakiety umieszczone w wiadrze opuszczają je następnie ze stałą prędkością Δp, dzięki czemu zostaje osiągnięty efekt, w którym pakiety są przesyłane dalej do sieci z prędkością nie większą niż pozwala na to ustalona wartość Δp. Jednak, jeśli nadchodzące pakiety są różnych rozmiarów, prędkość transmisji będzie ściśle uzależniona od rozmiaru tych pakietów. Mechanizm ten można przyrównać do wiadra z wodą, z którego przez dziurkę w spodzie równomiernie wyciekają krople wody.



http://en.wikipedia.org

Model „wiadra z żetonami” (Token Bucket)

Bardziej elastycznym mechanizmem jest model „wiadra z żetonami” (ang. Token Bucket), wykorzystujący do kształtowania ruchu model żetonu, który zazwyczaj reprezentuje jednostkę danych o określonym rozmiarze bajtów lub pakiet danych o ustalonym rozmiarze.



Przepływ żetonów jest definiowany przez dwie wartości: napływ żetonów r (ich produkcja) w stałym odstępie czasu Δt, oraz głębokość wiadra b, czyli ilość żetonów, jakie może ono pomieścić.

Jeśli „wiadro” jest pełne, nadchodzące żetony są odrzucane.

Każdy żeton umożliwia przesłanie określonej ilości bitów danych lub jeden pakiet danych. Aby pakiety zostały przesłane dalej, muszą one otrzymać jeden żeton na jeden pakiet, bądź ilość żetonów na jeden pakiet, adekwatną do rozmiaru informacji w nim zawartych. Mechanizm ten pozwala, więc na dokładne ustalenie ograniczenia prędkości transmisji w odniesieniu do napływających pakietów o różnej wielkości, na co nie pozwalał model „cieknącego wiadra”.

Jeżeli w wiadrze brakuje żetonów, pakiet jest buforowany do momentu pojawienia się odpowiadającej mu ilości żetonów, przy czym bufor ma swoją objętość m po przekroczeniu której, kolejne nadchodzące pakiety są odrzucane. Jeśli w wiadrze występuje nadwyżka żetonów, pakiety mogą przez chwilę być wysyłane nieco szybciej, niż wynika to z ustanowionego ograniczenia, wykorzystując tą chwilową nadwyżkę żetonów.

W praktycznych implementacjach często spotyka się oba te modele połączone. Najczęściej stosuje się model, w którym dane z „wiadra żetonów” TB trafiają do „cieknącego wiadra” LB, zwanym „wiadrem żetonów z kontrolą prędkości cieknącego wiadra” (Token Bucket with Leaky Bucket rate control).

Mechanizm ten ma za zadanie wygładzić chwilową nadwyżkę wysyłanych pakietów z „wiadra żetonów”. Przykładem algorytmu kolejkowania stosującego ten mechanizm jest bezklasowy algorytm **TBF** (Tocken Bucket Filter).

Token Bucket Filter (TBF)

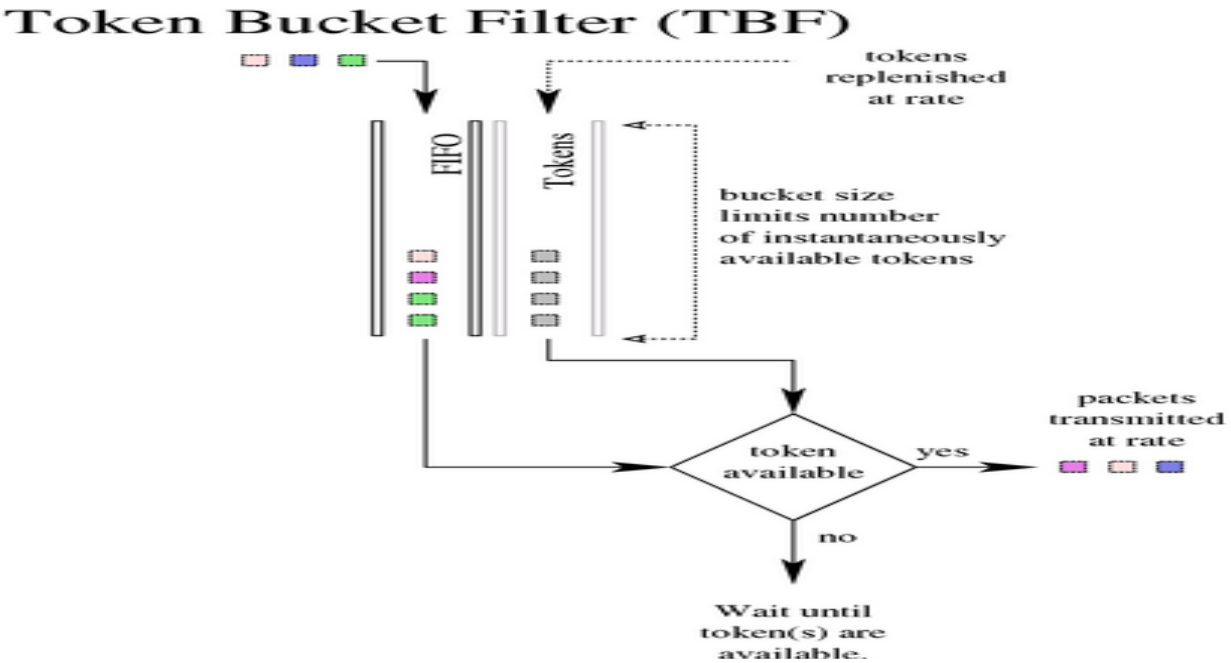
Bezklasowy algorytm TBF działa w oparciu o żetony (tokens) w wiadrze, w przeciwieństwie do SFQ, algorytm ten pozwala na ograniczenie pasma. Z wiadrem skojarzony jest generator żetonów, które trafiają do wiadra. Z każdym cyklem z wiadra pobierany jest żeton (lub żetony) oraz porcja bajtów do wysłania.

Możliwe są trzy sytuacje:

- pakiety nadchodzą z częstością generowania żetonów - wtedy pakiety dekolejkowane są bez opóźnień;

- pakiety nadchodzą z częstością mniejszą od generowania żetonów - pakiety są dekolejkowane bez opóźnień, co jakiś czas możliwe jest wysyłanie pakietów z większą prędkością w seriach (bursts);

- pakiety nadchodzą z częstością większą od żetonów - żetony w wiadrze się wyczerpią i pakiety będą opóźniane, aż w końcu odrzucane.



http://www.cs.put.poznan.pl/ddwornikowski/sieci/sieci2/ts.html

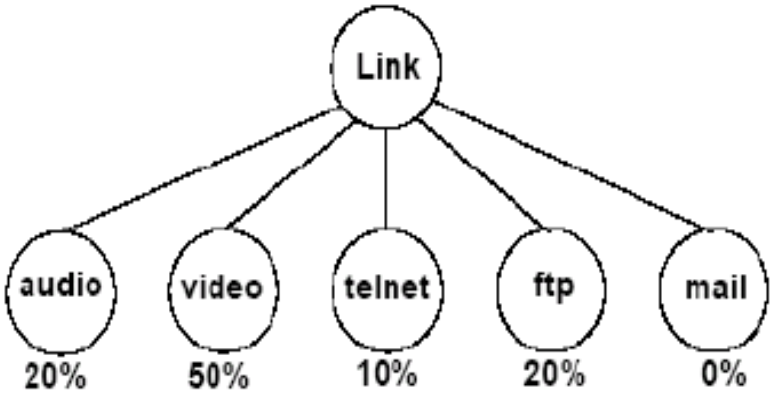
5.5.4. Algorytmy hierarchicznego podziału łącza

Algorytmy hierarchicznego podziału łącza mają na celu połączenie zalet wszystkich dotąd omawianych metod.

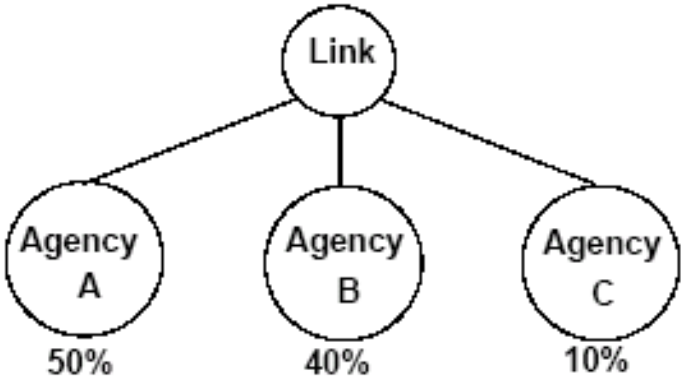
Pozwalają one na dokładnie określony podział parametrów jakości, przede wszystkim przepustowości łącza, pomiędzy zdefiniowane klasy ruchu.

Klasy ruchu

Klasa ruchu jest „pewnym agregatem ruchu grupującym pakiety razem dla celów jakiegoś sposobu zarządzania nimi". Klasy te mogą być wyodrębniane na podstawie różnych parametrów znajdujących się w nagłówkach protokołów, np. adres IP nadawcy lub odbiorcy, port nadawcy lub odbiorcy, itd. Przykładowo: może to być podział przepustowości na klasy usług, w którym ruch audio otrzymuje 20% prędkości łącza, ruch video 50%, ruch telnet 10%, ruch ftp 20%, a ruch usługi e-mail 0%.

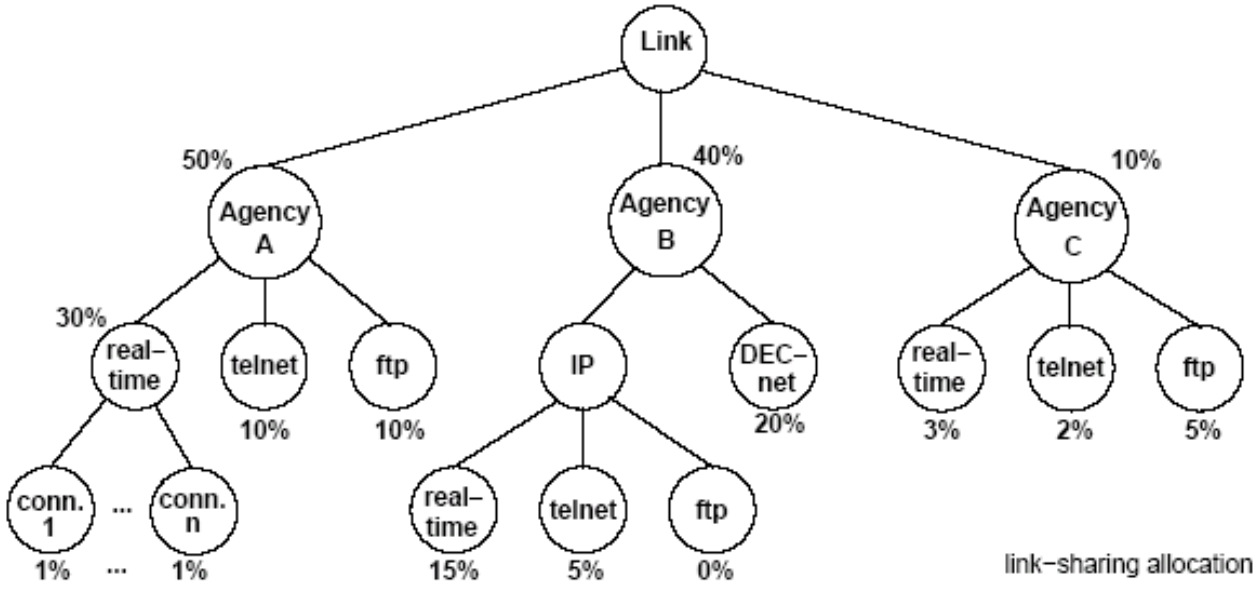


Klasy usług mogą wyznaczać nie tylko konkretne usługi, ale także może to być podział zasobów pomiędzy podsieci, czy konkretnych odbiorców lub nadawców. Przykładowo: może to być podział prędkości łącza na trzy podsieci, w którym podsieć „Agency A" otrzymuje 50% przepustowości, podsieć „Agency B" 40%, oraz podsieć „Agency C" 10%.



Algorytmy hierarchicznego podziału łącza umożliwiają podział na klasy ze względu bądź na usługi, bądź tylko odbiorców, bądź tylko nadawców. Największą ich zaletą jest jednak fakt, że mogą pozwolić na podział przepustowości najpierw według odbiorców, czy nadawców, a następnie w ramach ich wyodrębnionego ruchu jeszcze podział na konkretne usługi. Może to być także podział najpierw na całe podsieci, potem w ramach tych podsieci na konkretnych nadawców, lub odbiorców i w ich ramach jeszcze na ruch danych usług.

Przykładowo: algorytmy te mogą rozwiązać kwestie następującego podziału nadającego określoną gwarancję przepustowości. Całą przepustowość łącza należy najpierw podzielić na trzy podsieci, w stosunku podsieć „Agency A" 50%, „Agency B" 40%, a następnie „Agency C" 10%. W ramach podsieci „Agency A" ruch musi zostać rozłożony w stosunku z przydzielonych 50%, dla ruchu usług czasu rzeczywistego (ang. real-time) 30%, dla ruchu usługi telnet 10%, oraz dla ruchu usługi ftp 10%. Przydzielone 40% dla podsieci „Agency B" należy rozdzielić na kolejne dwie podsieci „IP" oraz „DECnet" po 20% dla każdej, a następnie z 20% dla podsieci „IP", 15% dla ruchu usług czasu rzeczywistego, 5% dla ruchu usługi telnet, oraz 0% dla ruchu usługi ftp. Dla podsieci „Agency C" z przydzielonych 10% z kolei, należy rozłożyć dla ruchu usług czasu rzeczywistego 3%, dla ruchu usługi telnet 2%, oraz dla ruchu usługi ftp 5% (rys. poniżej).



Struktura hierarchicznego podziału łącza ma charakter struktury drzewa. Na samej górze tej struktury znajduje się „pień” (ang. root), który reprezentuje całkowitą przepustowość łącza. W drzewie tym wyróżnia się następnie „klasy rodzice” (parent class), oraz „klasy dzieci” (child class). Związane jest to z faktem, że algorytmy hierarchicznego podziału łącza są algorytmami opartymi o klasy. W szczególności, klasa rodzic jest nadrzędna w stosunku do klas dzieci. Zawiera ona w sobie zgodnie z mechanizmem klasowych algorytmów kolejkowania danych, kolejne algorytmy kolejkowania, którymi są klasy dzieci. Klasa dziecko, może być, więc z kolei klasą rodzicem dla następnych zawartych w sobie algorytmów, jedynie „pień” nie posiada swojej klasy rodzica.

W algorytmach hierarchicznego podziału łącza następuje podział przepustowości klasy pnia na klasy rodzice, które następnie są dzielone na klasy dzieci, przy czym przepustowość tego podziału musi być sumarycznie nie większa niż posiada klasa rodzic.

Pakiety wygenerowane przez aplikacje użytkowników (ftp, telnet, real-time) są zbierane przez algorytm zarządzający pakietami, który tworzy kolejkę złożoną, zapewniającą podział przepustowości łącza zgodnie z ustaloną strukturą hierarchiczną.

Pożyczanie „przepustowości”

Jeśli jedna klasa nie wykorzystuje swojej przydzielonej przepustowości, w algorytmach hierarchicznego podziału łącza istnieje możliwość „pożyczenia” tych niewykorzystywanych zasobów swoim podklasom. W związku z tym mechanizmem, można wyróżnić statyczny oraz dynamiczny podział łącza. Statyczny podział nie uwzględnia możliwości oddania niewykorzystywanych zasobów innym klasom, natomiast dynamiczny podział taką sytuację umożliwia.

Do ostatniej klasy dziecka może być także podpięty jeszcze całkowicie inny algorytm kolejkowania, na przykład algorytm Fair Queuing, który zapewni równy podział pomiędzy sesje danych w ramach tej klasy dziecka.

Standardowo do ostatniej klasy dziecka podpinany jest algorytm „First In First Out”, jeśli nie zostanie w jego miejsce umieszczony inny.

Ze względu na największe możliwości z punktu widzenia obsługi użytkowników, algorytmy hierarchicznego podziału łącza są obecnie najczęściej stosowanymi podstawowymi algorytmami kolejkowania danych w sieciach LAN/WAN.

Najpopularniejszymi algorytmami hierarchicznego podziału łącza są:

- CBQ (ang. Class Based Queuing),

- HTB (ang. Hierarchical Token Bucket),

- HFSC (ang. Hierarchical Fair Service Curve),

- HPFQ (ang. Hierarchical Packet Fair Queuing).

CBQ (Class Based Queueing) jest algorytmem, stanowiącym podstawę do podziału przepustowości łącza (link sharing), oraz szkieletem pozwalającym na wykorzystanie bezklasowych algorytmów elementarnych. W CBQ ruch jest dzielony na kilka kolejek, zwanych dalej klasami, które charakteryzują się priorytetem oraz przydzieloną przepustowością. Siłą CBQ jest to, że pakiety mogą być rozdzielane do kolejek (klas) na podstawie dowolnych kryteriów za pomocą odpowiednich reguł (filtrów). Natomiast zamiast domyślnie ustawianych kolejek FIFO, składowymi CBQ mogą być dowolne z również opisanych wyżej algorytmów elementarnych. Cechy te pozwalają na osiągnięcie następujących efektów:

- podział sumarycznej przepustowości łącza na kilka części (klas), przydzielonych według potrzeb określonym rodzajom usług, adresom IP, itp.

- przesyłanie pakietów z różnym pierwszeństwem w zależności od tych samych parametrów;

- przydzielenie odpowiednim rodzajom ruchu właściwych algorytmów kolejkowania, np. dla ruchu masowego – algorytmu SFQ, dla ruchu TCP – algorytmu RED (Random Early Detection), umożliwiającego unikanie przeciążeń, itp.

HTB (ang. Hierarchical Token Bucket) jest klasowym (złożonym - hierarchicznym) algorytmem kolejkowania, który działa tak jak CBQ, ale nie wykonuje przeliczania czasu bezczynności. Zamiast tego funkcjonuje jako TBF z klasami, w którym zaimplementowano zaawansowany mechanizm pożyczania pasma. Każda podklasa może pożyczać wolne pasmo z nadklasy, gdy poziom jej dostępnego pasma spadnie poniżej progu minimalnego, gwarantowanego pasma (rate). Pożyczanie pasma następuje, aż do osiągnięcia pasma maksymalnego (ceil). W ten sposób możliwe jest zaimplementowanie złożonego mechanizmu dzielenia pasma „do” jakiejś wielkości, a jednocześnie istnieje możliwość zapewnienia pasma gwarantowanego. Kształtowanie ruchu zachodzi jedynie w liściach.

Pakiet wchodzący do złożonej kolejki jest klasyfikowany do określonej klasy przez filtr, kierujący się wybranymi parametrami pakietu. W zależności od rodzaju filtra mogą to być: adres źródłowy i docelowy pakietu, port, protokół, itp. Na przykład w linuxowej implementacji QoS dostępne są trzy podstawowe filtry – route, fw oraz u32. Do konfiguracji przepływu danych można użyć programu tc (traffic control) z pakietu iproute.

Do stworzenia przykładowego podziału łącza można wybrać algorytm HTB w połączeniu z SFQ i z filtrem u32. HTB ma możliwości podobne do CBQ tylko, że jest szybszy (szybciej wysyła pakiety i mniej obciąża serwer) oraz dokładniejszy, a ponadto znacznie łatwiejszy w konfiguracji – konfiguracja HTB dobrze się skaluje przy wzroście skomplikowania. SFQ sprawiedliwie obsługuje wszystkie połączenia danej klasy.

5.6. Algorytmy zapobiegania zatorom

Odrębną grupą algorytmów kolejkowania danych są algorytmy zapobiegania zatorom, których przedstawicielem jest algorytm „losowego wczesnego wykrywania” RED (Random Early Detection). Pozostałe istniejące algorytmy zapobiegania zatorom stanowią modyfikację tego mechanizmu.

Algorytmy szeregowania pakietów są rozpatrywane z punktu widzenia podziału zasobów w sytuacjach przeciążenia sieci i starają się zminimalizować jego skutki. Nadchodzące pakiety są w tych mechanizmach buforowane w powstałych kolejkach. Jeśli jednak nadmiar napływających danych jest długotrwały, pojemność kolejki jest niewystarczająca do zachowania wszystkich tych przychodzących pakietów, zatem nadmiarowa część musi zostać odrzucona. W przypadku sesji TCP spowoduje to ponowną retransmisję tychże pakietów, co może jeszcze wzmocnić zjawisko przeciążenia.

W porównaniu do algorytmów szeregowania pakietów, które zajmują się kwestią jak się zachować w sytuacjach przeciążenia sieci, algorytmy zapobiegania zatorom zajmują się kwestią jak się zachować, aby do tego przeciążenia nie dopuścić. Efekt ten jest osiągany poprzez ***losowe odrzucanie*** pakietów, mające na celu spowolnić transmisję danych wybranych sesji TCP. Takie spowolnienie transmisji jest możliwe tylko w przypadku ruchu TCP, który posiada zaimplementowane mechanizmy dostosowania prędkości przesyłania danych do warunków sieci.

RED (Random Early Detection) to algorytm mający na celu unikanie przeciążeń przez wykorzystanie mechanizmów istniejących już w protokole TCP. Algorytm RED „przewiduje” wystąpienie przeciążenia łącza i gubi pakiety, sygnalizując tym samym nadawcy, że powinien ograniczyć transmisję. W przypadku protokołu TCP jest to założenie jak najbardziej poprawne. Po zakończeniu przeciążenia TCP potrafi automatycznie powrócić do optymalnej prędkości wysyłania pakietów. W związku z tym RED znajduje zastosowanie jako algorytm w podklasach HTB lub CBQ, przeznaczonych dla ruchu TCP. Charakterystyczną cechą RED jest to, że w odróżnieniu od algorytmów kształtowania ruchu, próbuje on zapobiegać przeciążeniu zanim ono wystąpi, a nie tylko zminimalizować jego skutki.

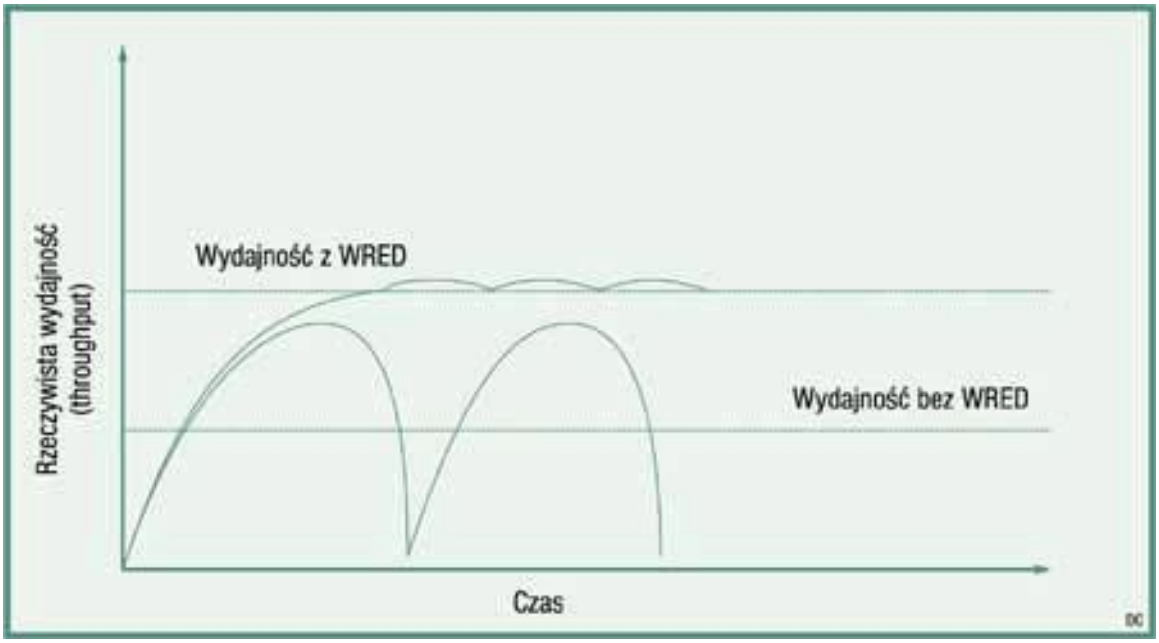
Zadania algorytmów unikania przeciążeń

Zadaniem algorytmów zapobiegania zatorom jest więc takie sterowanie buforem, aby nie mogła nastąpić sytuacja przeciążenia łącza, czyli sytuacja, gdy kolejny nadchodzący do bufora pakiet zostaje odrzucony ze względu na jego przepełnienie. Efekt ten jest osiągany poprzez ciągłe obliczanie średniej długości kolejki i porównywanie jej z dwoma progami: minimalnym i maksymalnym. Jeśli średnia wielkość kolejki znajduje się poniżej minimalnego progu wówczas ani jeden przybywający pakiet nie będzie odrzucony. Jeśli średnia wielkość kolejki znajduje się powyżej progu maksymalnego, wtedy każdy nowo przybyły pakiet zostaje odrzucony. Jeśli średnia znajduje się pomiędzy wartościami tych progów, wówczas pakiety są odrzucane bazując na obliczeniach prawdopodobieństwa dokonywanych na podstawie średniej wielkości kolejki. Czym bardziej więc wielkość kolejki zbliża się do wartości maksymalnego progu, tym więcej pakietów jest odrzucanych. Taki mechanizm powoduje, że połączenia zużywające aktualnie więcej przepustowości są bardziej narażone na odrzucanie ich pakietów.

Algorytmy zapobiegania zatorom są dużo bardziej przydatne w węzłach sieci szkieletowych, gdzie występuje transmisja danych o dużej przepustowości. Utworzony bufor dla tak dużego ruchu, musiałby być bardzo duży, aby nie powodować zbytniego odrzucania pakietów, a to z kolei powodowałoby znaczne zwiększenie opóźnienia pakietu. Efekt retransmisji danych ma także dużo większy wpływ przy dużych prędkościach przesyłania danych. Dodatkowo algorytmy te są w stanie poradzić sobie z „wybuchami” ruchu, czyli nagłym jego zwiększeniem, gdyż zaczynają one odrzucać pakiety zanim jeszcze bufor zostanie zapełniony, co ma dużo większe znaczenie przy dużych prędkościach w sieciach szkieletowych.

Najbardziej znaną odmianą mechanizmu RED jest algorytm „ważonego losowego wczesnego wykrywania” (ang. Weighted Random Early Detection). Algorytm ten potrafi rozpoznawać różne klasy ruchu, dla których z osobna są definiowane progi wypełnienia kolejek oraz maksymalne prawdopodobieństwo odrzucenia pakietu. Mechanizm ten pozwala na zapobieganie przeciążeniom, poprzez ograniczanie w pierwszej kolejności ruchu mniej istotnego.

Wpływ stosowania mechanizmu WRED na efektywną transmisję



5.7. Modele sieci komputerowych wspierających QoS

W ramach IETF (Internet Engineering Task Force) opracowano dwie odmienne architektury sieci (modele sieci) systemowo wspierających jakość usług.

Modele zapewniania QoS

- model oparty o architekturę **usług zintegrowanych** (IntServ).

- model oparty o architekturę (model) **usług zróżnicowanych** (DiffServ),

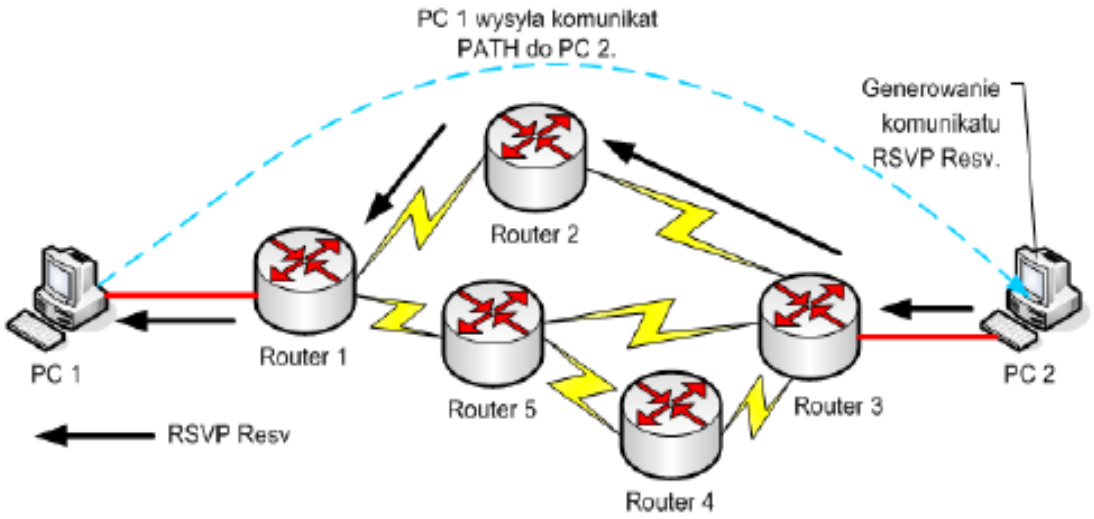
Integrated Services (usługi zintegrowane, w skrócie: IntServ) – jeden z mechanizmów, zapewniających jakość usług (ang. Quality of Service).

Zdefiniowała go w 1997 grupa robocza IETF Integrated Service w celu optymalizacji QoS (gdzie Internet Engineering Task Force (IETF) to nieformalne, międzynarodowe stowarzyszenie osób zainteresowanych ustanawianiem standardów technicznych i organizacyjnych w Internecie).

Model usług zintegrowanych Intserv (ang. Integrated Services) – charakteryzuje się rezerwacją zasobów dla przepływającego strumienia. Odpowiadają za to mechanizmy sygnalizujące, które informują routery na trasie datagramu o wymaganiach stawianych przez aplikacje nadające dane. Rezerwacja następuje tylko w przypadku, gdy router posiada odpowiednią ilość wymaganych zasobów.

Po dokonaniu rezerwacji przez wszystkie węzły na trasie strumienia danych następuje zestawienie połączenia i przesłanie datagramów. Do sygnalizacji w sieci wykorzystywany jest protokół RSVP (Resource ReSerVation Protocol) w celu sygnalizacji rezerwacji zasobów.

Przed rozpoczęciem transmisji nadawca (PC1) wysyła do odbiorcy (PC2) wiadomości typu PATH w celu wyznaczenia ścieżki przepływu danych. Następnie generowany jest komunikat protokołu RESV i przesyłany w kierunku odwrotnym - od odbiorcy do nadawcy, w celu dokonania rezerwacji w routerach zasobów (bufory, przepustowości łączy), gwarantujących wymaganą jakość usługi. Przy czym każdy węzeł znajdujący się na ścieżce od odbiorcy do nadawcy może zaakceptować lub odrzucić żądanie rezerwacji.



5.7.1. Cechy architektury IntServ

W architekturze IntServ (RFC1633) zakłada się, że zasoby w sieci są rezerwowane dla poszczególnych lub zagregowanych strumieni danych. W tym celu został zdefiniowany specjalny protokół sygnalizacyjny znany pod nazwą protokołu RSVP (Reservation Protocol) (RFC2205, RFC2210), który umożliwia realizację żądania przez daną aplikację, wymagającą rezerwacji zasobów w sieci.

Implementacja tego protokołu jest wymagana w każdym ruterze IP. W konsekwencji, ruter IP będzie przyjmował i realizował żądania rezerwacji, co wiąże się z przechowywaniem informacji o każdej poczynionej rezerwacji wraz z informacją o skojarzonym strumieniu danych.

Architektura usług zintegrowanych zapewnia pełne rozróżnienie pakietów pochodzących z różnych sesji. Pozwala to na odrębne traktowanie poszczególnych strumieni i zarazem zapewnienie jakości usługi w relacji od końca do końca (end-to-end). Dla każdego strumienia można zdefiniować odmienne wartości parametrów QoS. Ponadto strumienie danych są niezależne i nie zakłócają się wzajemnie.

W modelu IntServ zdefiniowano trzy klasy usług:

- usługę gwarantowaną GS (Guaranteed Service),
- usługę kontrolowanego obciążenia CL (Controlled Load Service),
- usługę niesklasyfikowaną BS (Best Effort).

W architekturze IntServ zdefiniowano, oprócz standardowej, niesklasyfikowanej usługi typu **Best Effort (BS)**, dwie dodatkowe usługi:

- **Guaranteed Service GS** (RFC 2212) – przeznaczoną dla aplikacji wymagających gwarancji odnośnie parametrów jakości przekazu danych związanych z opóźnieniami,

- **Controlled Load Service C**L (RFC 2211) – przeznaczoną dla aplikacji wymagających bezstratnego przekazu danych i charakteryzującą się jakością przekazu określaną jako lepszą niż Best Effort.

Klasa usług GS zapewnia gwarantowaną szerokość pasma transmisyjnego, brak strat pakietów wynikających z kolejkowania w buforach urządzeń sieciowych oraz nieprzekraczalną wartość opóźnienia przesyłanych pakietów w relacji od końca do końca. Brak jest natomiast gwarancji na nierównomierność opóźnienia. Klasa GS znajduje zastosowanie do aplikacji bardzo wrażliwych na opóźnienia i wymagających transmisji w czasie rzeczywistym (np. przesyłanie obrazu wideo).

Klasa usług CL nie zapewnia ścisłych gwarancji ilościowych parametrów QoS. Gwarantuje jedynie takie warunki ruchowe, jakie występowałyby w sieci nieobciążonej ruchem. Oznacza to, że w warunkach rosnącego obciążenia sieci jakość transmisji nie powinna się pogarszać. Klasa usług CL jest przeznaczona między innymi do aplikacji czasu rzeczywistego, ale tolerujących pewien poziom opóźnień i strat pakietów.

Usługa niesklasyfikowana BS jest zorientowana na klasyczną transmisję danych, bez żadnych gwarancji QoS. Wraz ze wzrostem obciążenia sieci poziom jakości transmisji znacznie się obniża. Klasa ta jest przeznaczona do aplikacji tolerujących duże opóźnienia oraz duże zmiany wartości opóźnienia.

Każdy węzeł sieci zgodny z modelem IntServ powinien mieć zaimplementowane wymienione poniżej następujące mechanizmy sterowania ruchem.

- Sterowanie przyjmowaniem zgłoszeń (admission control) – mechanizm polegający na sprawdzaniu, czy dostępne zasoby są wystarczające do zapewnienia żądanych parametrów transmisyjnych.

- Klasyfikacja ruchu (traffic classification) – rozróżnianie pakietów należących do różnych strumieni.

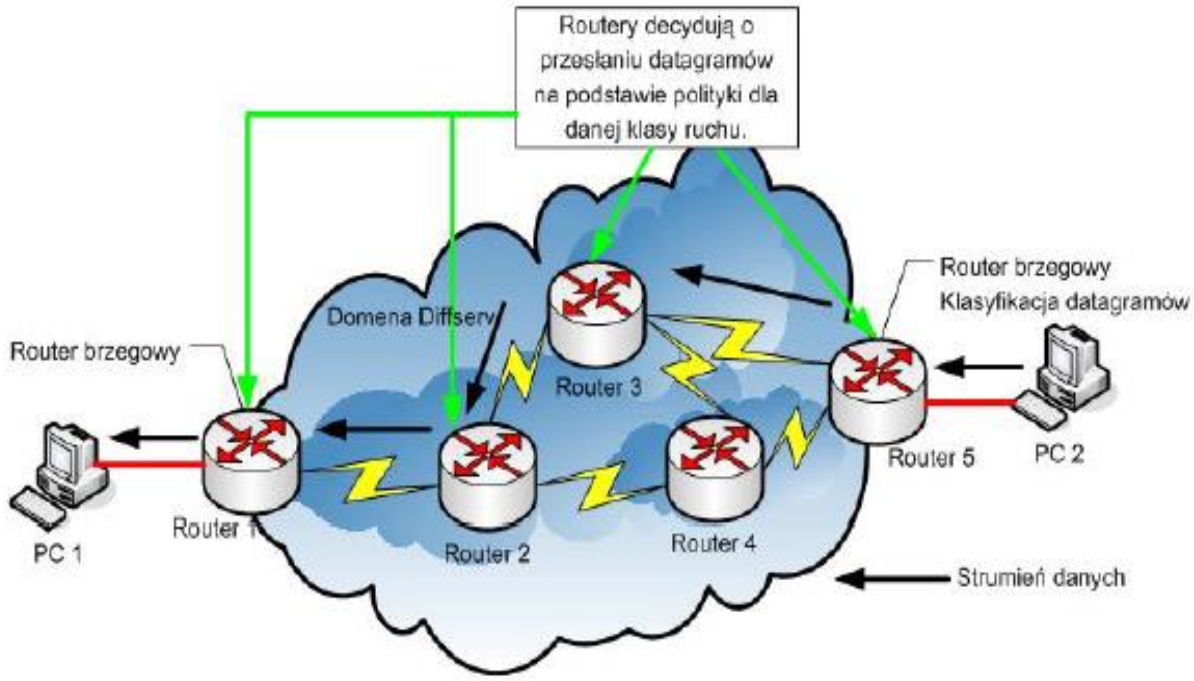
- Sprawdzanie zgodności charakterystyki ruchowej strumienia danych z kontraktem ruchowym (traffic policing).

- Szeregowanie pakietów (scheduling), polegające na zmianie kolejności pakietów w kolejce wyjściowej, mające na celu zapewnienie strumieniom danych gwarantowanej jakości obsługi.

5.7.2. Cechy architektury DiffServ

DiffServ (Differentiated Services) to architektura sieci pozwalająca na gwarantowanie jakości usług przesyłania danych (ang. QoS, Quality of Service) w sieciach IP.

Model usług zróżnicowanych DiffServ (ang. Differentiated Services) – charakteryzuje się dzieleniem przepływających datagramów na klasy i zapewnianie odpowiedniej obsługi w zależności od niej. Wszystkie routery w domenie DiffServ działają autonomicznie, opierając się na klasie przydzielonej strumieniowi danych i politykach ich obsługi tzw. PHB (ang. Per Hop Behaviour). Zaletą tego modelu jest duża skalowalność oraz elastyczność w porównaniu do modelu IntServ.



Celem stworzenia usług zróżnicowanych Diffserv przez IETF było obmyślenie skalowalnego mechanizmu klasyfikowania ruchu opartego o autonomiczne jednostki zagregowane w jednej domenie, które mogą działać niezależnie w oparciu o własne polityki dotyczące obsługi ruchu danej klasy. Jednym z głównych założeń w DiffServ było stworzenie alternatywnego mechanizmu do rozróżniania usług w Internecie, osłabiając problem skalowalności występujący w modelu IntServ.

Zapewnianie jakości usług nie odbywa się na poziomie pojedynczego połączenia, lecz na poziomie pewnej klasy (grupy) połączeń. Grupa robocza DiffServ zdefiniowała w nagłówku IP pole Differentiated Services (pole DS). Pole DS składa się z sześciu bitów, które są częścią nagłówka IP, formalnie znanego jako oktet ToS (ang. Type of Service). DiffServ wykorzystuje bity ToS w celu przenoszenia informacji o aplikacji. O atrakcyjności DiffServ i ToS decyduje możliwość zmiany bitów ToS w czasie wędrówki pakietu od węzła do węzła, zapobiegając sytuacji kiedy wszystkie aplikacje nadadzą sobie najwyższy priorytet – wtedy priorytety tracą swój sens.

Usługi DiffServ pozwalają w polu DS wskazać tylko skończoną liczbę klas usług. W modelu DiffServ maksymalna liczba klas usług różniących się parametrami QoS została ograniczona do 64.

Główną zaletą podejścia DiffServ jest skalowalność. Zasoby są przydzielane na podstawie klas, a liczba stanów (instancji) informacji rośnie wraz z liczbą klas, zaś wolniej wraz z ilością strumieni aplikacyjnych. Model DiffServ rozwiązuje problem zarządzania ruchem tylko na odcinku do następnego skoku. Model sterowania DiffServ składa się ze zbioru mechanizmów sterowania mikroinżynieriami ruchu (mikro-TE). Inne możliwości inżynierii ruchu, takie jak zarządzanie przepustowością (włączając w to sterowanie trasowaniem), są również wymagane w celu dostarczenia akceptowanej jakości usługi w sieci DiffServ.

Architektura DiffServ jest obecnie uważana za jedyną pozwalającą na zapewnienie ścisłych gwarancji QoS w sieci IP. Aby zapewnić QoS, stosowane są następujące mechanizmy:

- kształtowanie i ograniczane przepustowości,
- zapewnienie sprawiedliwego dostępu do zasobów,
- nadawanie odpowiednich priorytetów poszczególnym pakietom wędrującym przez sieć,
- zarządzanie opóźnieniami w przesyłaniu danych,
- zarządzanie buforowaniem nadmiarowych pakietów: DRR, WFQ, WRR,
- określenie charakterystyki gubienia pakietów,
- unikanie przeciążeń: Connection Admission Control (CAC), Usage Parameter Control (UPC).

Działanie modelu usług zróżnicowanych

Celem wprowadzenia modelu DiffServ w sieciach komputerowych jest zapewnienie odpowiedniej obsługi strumieni danych umożliwiając łatwe jej skalowanie. Niezawodne dostarczanie jakości usług w sieciach umożliwia mu rozbudowana architektura składająca się z wielu komponentów wykonujących wyspecjalizowane funkcje. W skład modelu DiffServ wchodzą m.in. mechanizmy:

- Klasyfikacji i kolejkowania pakietów.
- Oznaczania pakietów.
- Zarządzania zatorami.
- Unikania zatorów.

Obszar sieci, w której zaimplementowany jest model DiffServ określany jest mianem Domeny DiffServ (DS), składającej się z routerów brzegowych (odpowiedzialnych za klasyfikowanie strumieni danych) oraz routerów szkieletowych przesyłających pakiety zgodnie z własnymi politykami. Routery szkieletowe przesyłają strumienie danych zgodnie z zasadami określonymi przez PHB, czyli model określający sposób traktowania danej klasy ruchu rozpoznawanej na podstawie odpowiedniego pola w nagłówku IP.

Administratorzy mają do dyspozycji następujące rodzaje usług PHB:

- Przekazywanie przyśpieszone EF (ang. Expedited Forwarding) – określa, że dostarczony pakiet powinien zostać wysłany do kolejnego węzła z jak najmniejszym opóźnieniem. Dodatkowo pakiety należące do klasy ruchu o skonfigurowanej prędkości muszą zostać wysłane z taką samą lub wyższą prędkością.

- Przekazywanie gwarantowane AF (ang. Assured Forwarding) – model ten klasyfikuje każdy strumień do jednej z czterech klas ruchu, każda z nich uzyskuje część bufora oraz pasma. W obrębie klasy każdy pakiet otrzymuje jedną z trzech preferencji odrzucenia, określającą w jakiej kolejności router będzie odrzucał dane w przypadku wykorzystania zasobów przez daną klasę.

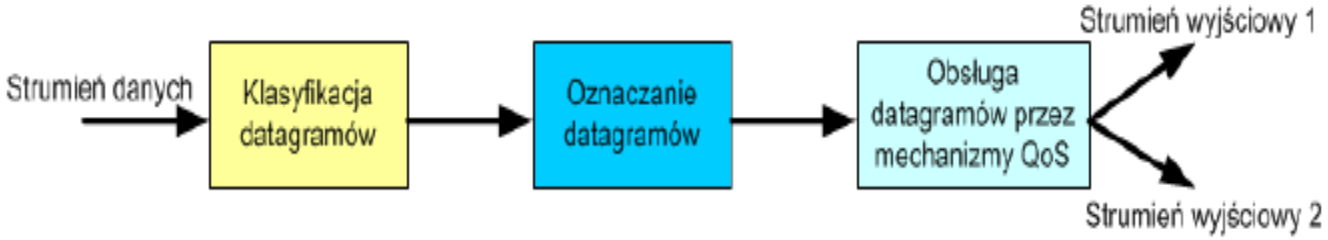
- Przekazywanie niesklasyfikowane (bez opcji klasyfikowania).

Klasa EF zapewnia małe opóźnienia pakietów oraz małą zmienność opóźnienia. Pakiety należące do tej usługi mają też zagwarantowane pewne pasmo. Usługa ta może służyć do tworzenia dedykowanych łączy punkt-punkt o określonych parametrach.

W przypadku klas usług z grupy AF nie ma gwarancji dotyczącej wielkości opóźnienia pakietów. Określa się jedynie, że ruch zgodny z ustalonym profilem będzie dostarczony z prawdopodobieństwem nie niższym niż ustalony próg. Zdefiniowano cztery klasy usług AF. W ramach każdej z nich określono trzy poziomy prawdopodobieństwa odrzucenia pakietu (drop precedence). Pakiety przyporządkowane do danej klasy AF są umieszczane we wspólnej kolejce, a ich kolejność nie może być zmieniana. Poziomy prawdopodobieństwa odrzucenia pakietu służą priorytetyzacji pakietów w kolejce. W sytuacji przeciążenia sieci w pierwszej kolejności z kolejki usuwa się pakiety o największej wartości prawdopodobieństwa odrzucenia. Jakość usług w ramach danej klasy AF zależy głównie od wielkości przydzielonych dla niej zasobów sieciowych.

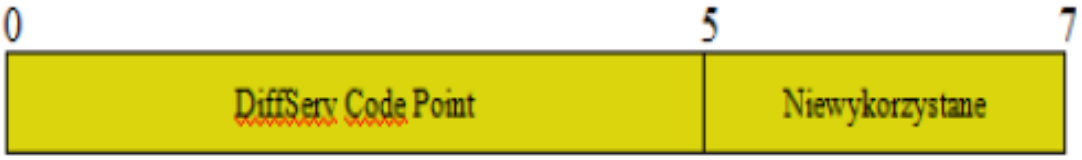
Klasyfikacja oraz oznaczanie pakietów

Klasyfikacja pakietów jest podstawą prawidłowego funkcjonowania modelu usług zróżnicowanych. Każdy strumień wchodzący do domeny DS przechodzi przez odpowiedni filtr klasyfikujący typ ruchu, następnie zostaje w odpowiedni sposób oznaczony. Każdy strumień oznaczony w ten sam sposób przydzielany jest do tej samej grupy oraz traktowany w ten sam sposób.

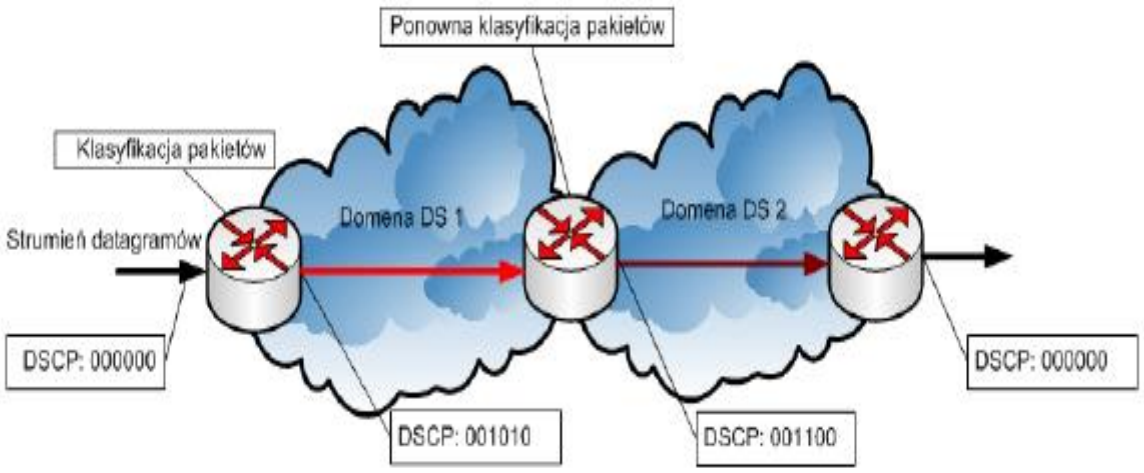


Przynależność pakietu do danej klasy usług w sieci DiffServ jest jednoznacznie określona przez wartość punktu kodowego DSCP (Differentiated Services Code Point). Przydzielenie pakietu do klasy usług odbywa się w wejściowym routerze brzegowym w chwili wejścia pakietu do domeny DiffServ.

Pole DS zbudowane jest z 6 bitowego pola DiffServ Code Point oraz 2 bitów nie wykorzystanych zarezerwowanych do późniejszego zastosowania.



Po przypisaniu pakietu do klasy usług nadawana jest mu odpowiednia wartość punktu kodowego. Wszystkie pakiety, mające taki sam punkt kodowy, tworzą zespół zachowań BA (Behaviour Aggregate). Pakiety te są jednakowo traktowane w ramach domeny. Sposób przesyłania pakietów określają reguły przesyłania pakietów PHB (Per Hop Behaviour). Wprowadzono pojęcie zbioru uporządkowanego reguł przesyłania pakietów – PSC (PHB Scheduling Class) – jest to zbiór reguł PHB powiązanych dodatkowym warunkiem, że pakiety podlegające tym regułom muszą być umieszczane w jednej kolejce, a ich kolejność nie może być zmieniana.



W sieci IP może istnieć wiele domen DiffServ, będących ściśle określonym zbiorem węzłów. Domeny są niezależne, ale mogą ze sobą współpracować. Obszar sieci, składający się z wielu ściśle ze sobą współpracujących domen, tworzy region. W każdej domenie DiffServ wyróżnia się węzły brzegowe i rdzeniowe. Wśród węzłów brzegowych można wyróżnić węzły wejściowe (przez które ruch wpływa do domeny) i wyjściowe (przez które ruch opuszcza domenę). Są one odpowiedzialne za wymuszenie odpowiednich charakterystyk ruchu wejściowego i wyjściowego.

Zarządzanie zatorami i unikanie zatorów

Ważnym aspektem modelu usług zróżnicowanych jest zarządzanie zatorami oraz unikanie zatorów w sieciach komputerowych. Zatory mogą doprowadzić do niekontrolowanego zmniejszenia efektywności działania sieci komputerowej, a nawet uniemożliwić jej funkcjonowanie.

Najpowszechniej stosowanym mechanizmem przeznaczonym do zarządzania zatorami wykorzystywanymi w QoS jest kolejkowanie datagramów w węzłach sieci.

Algorytmy stosowane do tego celu różnią się w zależności od potrzeb konkretnej sieci komputerowej (zależy od rodzaju ruchu obsługiwanego przez sieć).

Najczęściej stosowane mechanizmy kolejkowania zostały przedstawione w poprzednich punktach (algorytmy kolejkowania i zapobiegania zatorom).

5.7.3. Współpraca modeli IntServ i DiffServ

Model usług zintegrowanych IntServ oraz model usług zróżnicowanych DiffServ są technikami wzajemnie się uzupełniającymi. Można je stosować w różnych obszarach sieci.

- Protokół sygnalizacyjny RSVP, opracowany dla modelu IntServ, pozwala zapewniać żądaną jakość obsługi pojedynczym strumieniom danych w relacji od końca do końca. Nie może on być jednak stosowany w dużych sieciach ze względu na omówione wcześniej problemy skalowalności.

- Model DiffServ, dzięki ograniczeniu liczby klas usług oraz rezygnacji z protokołu sygnalizacyjnego, nie ma problemów ze skalowalnością i dobrze nadaje się do stosowania w dużych sieciach. Nie zapewnia jednak pełnego rozróżnienia pakietów pochodzących z różnych aplikacji, uniemożliwia to pełne ilościowe zagwarantowanie parametrów transmisji dla poszczególnych strumieni ruchu.

Korzystając z najważniejszych zalet modeli IntServ i DiffServ opracowano podstawowe zasady ich współpracy. Zakłada się, że model IntServ będzie stosowany w obszarze sieci dostępowej, obsługującej stosunkowo niewielką liczbę użytkowników, a zatem umiarkowaną liczbę pojedynczych strumieni danych wymagających rezerwacji zasobów sieciowych. Z kolei model DiffServ, obsługujący ruch zagregowany, będzie stosowany w sieciach szkieletowych jako element pośredniczący w zapewnieniu jakości usług w relacji od końca do końca.